

Lenguaje de Programación **JavaScript** básico

The image shows the JavaScript logo, which consists of the letters 'JS' in a bold, black, sans-serif font. The logo is centered within a solid yellow square.

Contenido

1.	Conceptos básicos.....	4
1.1.	Breve historia	4
1.2.	Dónde ejecutar código JavaScript	5
1.3.	Sintaxis	5
1.4.	Limitaciones	6
1.5.	Cómo ejecutar código JavaScript	6
2.	Gramática general	9
3.	Comentarios. JSDoc	9
4.	Variables. Ámbitos de utilización.	10
4.1.	Ámbito de las variables.....	10
5.	Uso de la consola del navegador	12
6.	Tipos de datos básicos.	12
7.	Tipos de datos especiales: null, undefined e Infinity	13
7.1.	Undefined.....	13
7.2.	Null	14
7.3.	Infinity	14
7.4.	Comparación entre null, undefined e Infinity	14
8.	Conversión entre tipos de datos.	14
9.	Literales.	15
9.1.	Arrays de literales.	16
9.2.	Literales lógicos booleanos.	16
9.3.	Literales enteros	16
9.4.	Literales de coma flotante.....	17
9.5.	Literales de objetos.	17
9.6.	Literales de cadenas de caracteres (string).	18
9.7.	Uso de caracteres especiales en cadenas de caracteres.	18
9.8.	Caracteres de escape.....	19
10.	Operadores.	19
10.1.	Operadores de comparación.	20
10.2.	Operadores aritméticos.....	21
10.3.	Operadores de asignación.....	22
10.4.	Operadores booleanos.	23
10.5.	Operadores bit a bit.	24
10.6.	Operador Pipe (>)	25
10.7.	Operadores de objeto.	25

10.8.	Otros operadores.	27
11.	Bloques de código.	28
12.	Decisiones.	28
12.1.	Construcción if.	29
12.2.	Construcción if... else.	29
12.3.	Construcción switch.	30
13.	Bucles.	30
13.1.	Bucle for.	30
13.2.	Bucle for.. in	31
13.3.	Bucle for.. of.	31
13.4.	Bucle while.	32
14.	Sentencias break y continue.	33
15.	Funciones	33
15.1.	Argumentos y valores de retorno.	34
15.2.	Argumentos de una función: Array arguments	35
15.3.	Parámetro REST y operador SPREAD	36
15.4.	Generadores	36
15.5.	Funciones anónimas.	37
15.6.	Funciones arrow	38
16.	Depuración de código JavaScript desde el navegador	39
16.1.	Abrir la consola de JavaScript	39
16.2.	Visualizar los errores del código	40
16.3.	Comprobar el tráfico de red	40
16.4.	Escribir código JavaScript	41
16.5.	Depurar nuestro código javascript	41
16.6.	Puntos de parada (breakpoints):	42
16.7.	Alternativa a las Dev Tools	43
17.	Ejecución de código JavaScript que accede a HTML	43
18.	Try, catch(), finally, throw.	45
	ANEXO I: Funciones y propiedades básicas de JavaScript	46
	<i>Funciones útiles para cadenas de texto</i>	46
	<i>Funciones útiles para arrays</i>	48
	<i>Funciones útiles para números</i>	50
	<i>Funciones útiles para mostrar valores en un documento HTML</i>	51
	ANEXO II: Conjuntos en JavaScript.	51
	Anexo II: tipo Symbol.	53

1. Conceptos básicos

JavaScript es un lenguaje de programación que se utiliza principalmente para crear páginas web dinámicas.

Una página web dinámica es aquella que incorpora efectos como texto que aparece y desaparece, animaciones, acciones que se activan al pulsar botones y ventanas con mensajes de aviso al usuario.

Técnicamente, JavaScript es un lenguaje de programación interpretado, por lo que no es necesario compilar los programas para ejecutarlos.

A pesar de su nombre, JavaScript no guarda ninguna relación directa con el lenguaje de programación Java.

JavaScript es un lenguaje con las siguientes características:

- **Ligero:** diseñado para ocupar poco espacio en memoria, fácil de implementar y de aprender.
- **Interpretado:** se lee y se ejecuta cada línea de código de forma secuencial.
- **Basado en prototipos:** es una variante de la programación orientada a objetos en la que los objetos no se crean instanciando clases sino clonando otros objetos o creándolos directamente. **¡En JavaScript todo es un objeto!**
- **Case sensitive:** sensible a mayúsculas y minúsculas.
- **Débilmente tipado.**
- **Monohilo:** un único hilo de ejecución se encarga de realizar el trabajo de interpretación del código, de forma que se dificultan los interbloqueos y se favorece el uso de las **llamadas asíncronas**.
- **Dinámico:** es capaz de cambiar importantes características del programa, como la estructura de un objeto o el tipo de una variable, mientras se está ejecutando. *¿Espera, espera, eso es bueno o malo?*
- **Programación orientada a objetos:** también implementa clases que se pueden instanciar para crear nuevos objetos que se pueden relacionar entre sí. **Recuerda: polimorfismo, herencia y abstracción.**

1.1. Breve historia

A principios de los años 90, la mayoría de usuarios que se conectaban a Internet lo hacían con módems a una velocidad máxima de 28.8 kbps. En esa época, empezaban a desarrollarse las primeras aplicaciones web y por tanto, las páginas web comenzaban a incluir formularios complejos.

Con unas aplicaciones web cada vez más complejas y una velocidad de navegación tan lenta, surgió la necesidad de un lenguaje de programación que se ejecutara en el navegador del usuario. De esta forma, si el usuario no rellenaba correctamente un formulario, no se le hacía esperar mucho tiempo hasta que el servidor volviera a mostrar el formulario indicando los errores existentes.

Brendan Eich, un programador que trabajaba en Netscape, pensó que podría solucionar este problema adaptando otras tecnologías existentes (como *ScriptEase*) al navegador Netscape Navigator 2.0, que iba a lanzarse en 1995. Inicialmente, Eich denominó a su lenguaje *LiveScript*.

Posteriormente, Netscape firmó una alianza con Sun Microsystems para el desarrollo del nuevo lenguaje de programación. Además, justo antes del lanzamiento Netscape decidió cambiar el nombre por el de JavaScript. La razón del cambio de nombre fue exclusivamente por marketing, ya que Java era la palabra de moda en el mundo informático y de Internet de la época.

La primera versión de JavaScript fue un completo éxito y Netscape Navigator 3.0 ya incorporaba la siguiente versión del lenguaje, la versión 1.1. Al mismo tiempo, Microsoft lanzó JScript con su navegador Internet Explorer 3. JScript era una copia de JavaScript al que le cambiaron el nombre para evitar

problemas legales.

Para evitar una guerra de tecnologías, Netscape decidió que lo mejor sería estandarizar el lenguaje JavaScript. De esta forma, en 1997 se envió la especificación JavaScript 1.1 al organismo ECMA (**European Computer Manufacturers Association**).

ECMA creó el comité TC39 con el objetivo de **"estandarizar un lenguaje de script multiplataforma e independiente de cualquier empresa"**. El primer estándar que creó el comité TC39 se denominó *ECMA-262*, en el que se definió por primera vez el lenguaje ECMAScript.

Por este motivo, algunos programadores prefieren la denominación **ECMAScript** para referirse al lenguaje JavaScript. De hecho, JavaScript no es más que la implementación que realizó la empresa Netscape del estándar ECMAScript.

La organización internacional para la estandarización (ISO) adoptó el estándar [ECMA-262](#) a través de su comisión IEC, dando lugar al estándar ISO/IEC-16262.

1.2. Dónde ejecutar código JavaScript

- **Lado del cliente:** proporciona objetos para gestionar el navegador y el modelo de objetos del documento (DOM) del mismo. El DOM es la estructura interna que entiende JavaScript cuando lee un documento HTML. De esta forma se puede controlar y responder a los eventos del usuario (clic del ratón, pulsación de una tecla, ...), formularios, acceso y modificación del estilo del HTML, navegación entre páginas o cualquier otro elemento de la interfaz de usuario.
- **Lado del servidor:** amplía el núcleo del lenguaje extendiéndolo con objetos de interés para poder ser ejecutado en un servidor, normalmente a través de [Node.js](#) o [Deno.js](#). Así, es posible comunicarse con una base de datos, gestionar ficheros del servidor, ofrecer respuestas a solicitudes de aplicaciones, ...

1.3. Sintaxis

La sintaxis de un lenguaje de programación se define como el conjunto de reglas que deben seguirse al escribir el código fuente de los programas para considerarse como correctos para ese lenguaje de programación.

La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C. Las normas básicas que definen la sintaxis de JavaScript son las siguientes:

- **No se tienen en cuenta los espacios en blanco y las nuevas líneas:** como sucede con XHTML, el intérprete de JavaScript ignora cualquier espacio en blanco sobrante, por lo que el código se puede ordenar de forma adecuada para entenderlo mejor (tabulando las líneas, añadiendo espacios, creando nuevas líneas, etc.)
- **Case sensitive:** *se distinguen las mayúsculas y minúsculas*: al igual que sucede con la sintaxis de las etiquetas y elementos XHTML. Sin embargo, si en una página XHTML se utilizan indistintamente mayúsculas y minúsculas, la página se visualiza correctamente, siendo el único problema la no validación de la página. En cambio, si en JavaScript se intercambian mayúsculas y minúsculas el script no funciona.
- **No se define el tipo de las variables:** al crear una variable, no es necesario indicar el tipo de dato que almacenará. De esta forma, una misma variable puede almacenar diferentes tipos de datos durante la ejecución del script.
- **No es necesario terminar cada sentencia con el carácter de punto y coma (;):** en la mayoría de lenguajes de programación, es obligatorio terminar cada sentencia con el carácter “;”. Aunque JavaScript no obliga a hacerlo, es conveniente seguir la tradición de terminar cada sentencia con el carácter del punto y coma (;).

▪ **Se pueden incluir comentarios:** los comentarios se utilizan para añadir información en el código fuente del programa. Aunque el contenido de los comentarios no se visualiza por pantalla, si que se envía al navegador del usuario junto con el resto del script, por lo que es necesario extremar las precauciones sobre la información incluida en los comentarios.

JavaScript define dos tipos de comentarios: los de una sola línea y los que ocupan varias líneas.

Ejemplo de comentario de una sola línea:

```
// a continuación se muestra un mensaje  
alert("mensaje de prueba");
```

Ejemplo de comentario de varias líneas:

```
/* Los comentarios de varias líneas son muy útiles cuando se necesita incluir  
bastante información en los comentarios */  
alert("mensaje de prueba");
```

1.4. Limitaciones

JavaScript fue diseñado de forma que se ejecutara en un entorno muy limitado que permitiera a los usuarios confiar en la ejecución de los scripts.

De esta forma, los scripts de JavaScript no pueden **comunicarse con recursos que no pertenezcan al mismo dominio** desde el que se descargó el script. Los scripts tampoco **pueden cerrar ventanas que no hayan abierto esos mismos scripts**. Las ventanas que se crean **no pueden ser demasiado pequeñas ni demasiado grandes ni colocarse fuera de la vista del usuario** (aunque los detalles concretos dependen de cada navegador).

Además, los scripts **no pueden acceder a los archivos del ordenador** del usuario (ni en modo lectura ni en modo escritura) y tampoco pueden **leer o modificar las preferencias del navegador**.

Por último, si la ejecución de un script dura demasiado tiempo (por ejemplo, por un error de programación) el navegador informa al usuario de que un script está consumiendo demasiados recursos y le da la posibilidad de detener su ejecución.

A pesar de todo, existen alternativas para poder saltarse algunas de las limitaciones anteriores. La alternativa más utilizada y conocida consiste en firmar digitalmente el script y solicitar al usuario el permiso para realizar esas acciones.

1.5. Cómo ejecutar código JavaScript

Para que podamos desarrollar aplicaciones web que se ejecuten en el lado del cliente lo primero que debemos saber es que el documento base estará escrito en HTML (o XHTML en su defecto). En esta sección nos centraremos en algunas de las formas que un desarrollador de páginas web tiene a su disposición a la hora de integrar código de scripting en documentos HTML. Las particularidades de otras tecnologías se verán en los capítulos correspondientes.

La integración de Javascript y HTML/XHTML podemos hacerla de diferentes formas. Esta flexibilidad se refleja en, al menos, tres formas de incluir código Javascript en páginas web.

JAVASCRIPT EN EL MISMO DOCUMENTO HTML

HTML se basa en el uso de unas etiquetas predefinidas para marcar el texto. Una de estas etiquetas es `<script>` (y `</script>` para indicar la finalización de un bloque de código embebido). Ésta etiqueta **puede incluirse en cualquier parte del documento**, aunque se recomienda que el código Javascript, salvo para propósitos concretos de generación de contenido a visualizar, se defina dentro de la cabecera del

documento HTML (entre las etiquetas <head>y </head>). Podemos ver un ejemplo a continuación:

```
<html>

<head>
  <meta http-equiv="Content-Type" content="text/HTML; charset="utf-8" />
  <title>Ejemplo 1</title>
  <script>
    alert("Prueba de JavaScript");
  </script>
</head>

<body>
  <h1>Ejemplo 1: código embebido</h1>
</body>

</html>
```

Esta técnica suele utilizarse cuando se definen instrucciones que se referenciarán desde cualquier parte del documento o cuando se definen funciones con fragmentos de código genéricos. La mayor desventaja de este método es que, si ese fragmento de código lo queremos utilizar en otras páginas, debemos incluirlo en cada una de ellas, lo cual es un inconveniente cuando tenemos que realizar modificaciones de dicho código.

JAVASCRIPT EN ELEMENTOS HTML

El último método suele utilizarse habitualmente como forma de controlar los eventos que suceden asociados a un elemento HTML concreto (aunque también puede utilizarse con otros fines). Consiste en insertar fragmentos de Javascript dentro de atributos de etiquetas HTML de la página:

```
<html>

<head>
  <meta http-equiv="Content-Type" content="text/HTML; charset="UTF-8" />
  <title>Ejemplo 3</title>
</head>

<body>
  <p onclick="alert('Prueba de JavaScript');"> Ejemplo 3: código en
  atributos.</p>
</body>

</html>
```

La principal desventaja es que el código Javascript se intercala con el código HTML y, dependiendo de la complejidad y longitud de éste, el mantenimiento y modificación del código puede resultar más

complicado.

JAVASCRIPT EN UN ARCHIVO EXTERNO

Las mismas instrucciones de Javascript que se incluyen entre un bloque `<script></script>` pueden almacenarse en un fichero externo con extensión `.js`. Al igual que sucede con los documentos HTML, los ficheros `.js` pueden crearse con cualquier editor de texto. A continuación, se muestra el contenido de un fichero externo que contiene código Javascript.

Archivo mensaje.js:

```
alert("Prueba de JavaScript");
```

La forma de acceder y enlazar esos ficheros `.js` con el documento HTML/XHTML es a través de la propia etiqueta `<script>`. No existe un límite en el número de ficheros `.js` que pueden enlazarse en un mismo documento HTML/ XHTML. El siguiente ejemplo muestra cómo se enlazaría un documento HTML/XHTML con el fichero anterior `mensaje.js`:

```
<html>

<head>
  <meta http-equiv="Content-Type" content="text/HTML; charset="utf-8" />
  <title>Ejemplo 2</title>
  <script src="js/mensaje.js"></script>
</head>
<body>
  <h1>Ejemplo 2: Fichero externo.</h1>
</body>

</html>
```

Al igual que en el ejemplo anterior, para que esta técnica funcione tenemos que incluir el atributo **src**. Este atributo contendrá un valor que indicará la ruta relativa (con respecto al fichero principal HTML/XHTML) de la ruta donde se encuentra el archivo Javascript que se quiere enlazar. En este caso está dentro de una carpeta denominada `js`. La única restricción de la etiqueta `<script>` es que solo puede enlazarse un archivo en cada etiqueta. Podemos incluir el número de etiquetas `<script>` que necesitamos.

Entre las ventajas de este método está que la vinculación de un mismo fichero externo puede hacerse desde varios documentos HTML/XHTML distintos. De esta forma, en el caso de que haya que modificar algo, solo hay que hacerlo en un único fichero. Cualquier modificación realizada en el fichero externo se ve reflejada inmediatamente en todas las páginas que lo enlacen.

Podemos comprobar si el navegador en el que se ejecuta nuestro código tiene activa la ejecución de código JavaScript y así, avisar al usuario de tal caso. Para avisar al usuario de que no tiene activo el permiso de ejecución de código JavaScript podemos introducir las etiquetas: `<NOSCRIPT></NOSCRIPT>`.

2. Gramática general

Hay ciertas recomendaciones generales en los patrones de diseño para que sean más eficientes, entendibles y fáciles de mantener y actualizar:

- Seguir manuales de buenas prácticas.
- Reducir el tamaño de las funciones al máximo.
- No repetirse (en inglés, Don't Repeat Yourself, DRY).
- Escribir con consistencia desde el principio.
- Los comentarios, como JSDoc.
- Respetar y restringir el ámbito (scope) de las variables.
- Hacer muchas pruebas, y automatizarlas.

3. Comentarios. JSDoc.

JSDoc es un generador de documentación API para JavaScript, similar a Javadoc o phpDocumentor. Agrega comentarios de documentación directamente a tu código fuente, junto al código mismo. La herramienta JSDoc escaneará tu código fuente y generará un sitio web de documentación HTML.

Puedes ver la documentación completa en : <https://jsdoc.app/>.

El propósito de JSDoc es documentar la API de una aplicación o biblioteca JavaScript. Se supone que querrás documentar elementos como módulos, espacios de nombres, clases, métodos, parámetros de métodos, etc.

Los comentarios JSDoc generalmente deben colocarse inmediatamente antes del código que se documenta. Cada comentario debe comenzar con una secuencia `/**` para que el analizador JSDoc lo reconozca. Se ignorarán los comentarios que comiencen con `/*`, `/**` o más de 3 estrellas. Esta es una característica que le permite suprimir el análisis de bloques de comentarios.

Ejemplos básicos:

```
/** Descripción simple de la función */  
function foo() {  
}
```

```
/**  
 * Representa un libro.  
 * @param {string} title – Título del libro..  
 * @param {string} author – Autor del libro.  
 * @returns {number}  
 */  
function Book(title, author) {  
    ...  
}
```

Especificamos los parámetros de las funciones, tipo y descripción. También especificamos el valor devuelto.

4. Variables. Ámbitos de utilización.

La forma más conveniente de trabajar con datos en un script es asignando primeramente los datos a una variable. Es incluso más fácil pensar que una variable es un cajón que almacena información. El tiempo que dicha información permanecerá almacenada, dependerá de muchos factores. En el momento que el navegador limpia o recarga la ventana o marco, cualquier variable conocida será eliminada.

Dispones de dos maneras de crear variables en JavaScript: una forma es usar la palabra reservada **var** seguida del nombre de la variable. Por ejemplo, para declarar una variable edad, el código de JavaScript será:

```
var edad;
```

Otra forma consiste en crear la variable, y asignarle un valor directamente durante la creación:

```
var edad = 38;
```

o bien, podríamos hacer:

```
var edad;
edad = 38; // Ya que no estamos obligados a declarar la variable
var altura, peso, edad; // Para declarar más de una variable.
```

La palabra reservada **var** se usa para la declaración o inicialización de la variable en el documento.

Una variable de JavaScript podrá almacenar diferentes tipos de valores, y una de las ventajas que tenemos con JavaScript es que no tendremos que decirle de qué tipo es una variable u otra.

4.1. Ámbito de las variables.

A la hora de dar nombres a las variables, tendremos que poner nombres que realmente describan el contenido de la variable. No podremos usar palabras reservadas, ni símbolos de puntuación en el medio de la variable, ni la variable podrá contener espacios en blanco. Los nombres de las variables han de construirse con caracteres alfanuméricos y/o el carácter subrayado (_). No podremos utilizar caracteres raros como el signo +, un espacio, %, etc. en los nombres de variables, y estos nombres no podrán comenzar con un número.

Si queremos nombrar variables con dos palabras, tendremos que separarlas con el símbolo "_" o bien diferenciando las palabras con una mayúscula, por ejemplo:

```
var mi_peso;
var miPeso; // Esta opción es más recomendable
```

Deberemos tener cuidado también en no utilizar nombres reservados como variables. Por ejemplo, no podremos llamar a nuestra variable con el nombre de **return** o **for**.

Podemos declarar variables locales dentro de un bloque de código con el uso de la palabra reservada **let**.

Ejemplo:

```
var a = 3;
var b = 10;
```

```
if (a > 1) {  
  var a = 5;  
  let b = 5;  
  console.log("Dentro del IF: "+a+ " - "+b);  
}  
  
console.log("Fuera del IF: "+a+ " - "+b);
```

Otro ejemplo:

```
for (let i; i < nombres.length; i++)  
{  
  alert("Nombre actual: "+nombres[i]);  
}
```

En el ejemplo anterior la variable "b" solo existirá dentro del bloque de la condición de la sentencia if. Fuera de él no existirá, en su lugar existirá la variable **b** declarada fuera con el valor **10**. Sin embargo, la variable **a** machaca su valor al volver a declararla con **var** dentro del bloque.

También podremos declarar variables globales en cualquier parte del código. Esto significa que podrá ser accesible desde cualquier parte. Para ello basta con declarar la variable sin usar "var" o "let". Aunque se puede hacer, su uso **no es nada recomendable**, pero si se deben usar variables globales, es recomendable declararlas siempre al principio del fichero JavaScript para conocer qué variables globales tenemos y sus nombres.

Uno de los problemas que nos podemos encontrar es: ¿qué ocurre si tenemos dos variables, una local y otra global, con el mismo nombre? En este caso prevalecería el valor de las variables locales, pero solamente dentro del bloque o función.

Ejemplo:

```
var mensaje = "gana la de fuera";  
  
function muestraMensaje() {  
  var mensaje = "gana la de dentro";  
  alert(mensaje);  
}  
  
alert(mensaje);  
muestraMensaje();  
alert(mensaje);
```

¿Qué se mostrará por pantalla?

¿Qué sucede si dentro de una función se define la variable global con el mismo nombre que otra variable global que ya existe? En este otro caso, la variable global definida dentro de la función simplemente modifica el valor de la variable global definida anteriormente.

Ejemplo:

```
var mensaje = "gana la de fuera";  
  
function muestraMensaje() {  
  mensaje = "gana la de dentro"; // No ponemos var.  
  alert(mensaje);  
}
```

```
}  
  
alert(mensaje);  
muestraMensaje();  
alert(mensaje);
```

¿Qué se mostrará por pantalla?

De forma similar podemos usar la palabra reservada **const** para declarar una constante cuyo valor no podrá variar. Se comporta como una variable declarada con **let**, es decir, no existe fuera del bloque en el que se declara.

5. Uso de la consola del navegador

A partir de ahora vamos a empezar a escribir código JavaScript. Antes de empezar, es útil que conozcas el uso de la consola del navegador. A través de ella vamos a poder visualizar valores, poner mensajes o empezar a depurar nuestro código. Es muy útil.

El uso básico de la consola consiste en usar el objeto “**Console**”. Su método más usado es `console.log(“mensaje”)`, esto visualizará el mensaje en la consola del navegador.

Nota: Para poder ver los mensajes de la consola tendrás que pulsar *F12* o *Ctrl+Mayús+i* en el navegador.

Algunos otros métodos útiles son:

- `console.dir()` → Muestra un listado interactivo de las propiedades de un objeto JavaScript específico. Esta lista le permite examinar el contenido de los objetos hijos.
- `console.error()` → Muestra un mensaje de error. Se pueden utilizar sustituciones de cadena y argumentos adicionales con este método.
- `console.info()` → Muestra un mensaje de información en la Consola Web. Se puede utilizar sustitución de cadena y argumentos adicionales con este método.
- `console.warn()` → Muestra un mensaje de advertencia. Se puede utilizar sustitución de cadena y argumentos adicionales con este método.

[Aquí](#) tienes más métodos que puedes usar en la consola.

6. Tipos de datos básicos.

Las variables en JavaScript podrán contener cualquier tipo de dato. A continuación, se muestran los tipos de datos soportados en JavaScript:

Tabla de tipos soportados por JavaScript

Tipo	Ejemplo	Descripción
string	“Hola Mundo”	Una serie de caracteres dentro de comillas dobles.
number	9.45 123_345.45	Un número sin comillas dobles. Podemos usar “_” para separar los miles de la parte entera. Usaremos

		“.” o “,” (coma) para separar la parte decimal.
BigInt	<pre> console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991 const grande = 123456789012345678901234567890n; // n al final para definir tipo BigInt const grande = BigInt("123456789012345678901234567890"); // Otra forma </pre>	Permite representar números enteros muy grandes —más grandes que el límite de los números estándar (Number).
boolean	True, false	Un valor verdadero o falso
object	<pre> var myObject = new Object({ name: "Pepe", age: 25, superaCurso: true }) // Otra forma: var myObject = { name: "Pepe", age: 25, superaCurso: true } </pre>	Es un objeto software que se define por sus propiedades y métodos.
function	<pre> function calcula(a, b, func) { return func(a, b); } function suma (a, b) { return a + b; } console.log("Resultado: " + calcula(7, 8, suma)); </pre>	La definición de una función

7. Tipos de datos especiales: null, undefined e Infinity

Estos tres valores son **tipos primitivos especiales** en JavaScript. Vamos uno por uno:

7.1. Undefined

undefined representa el **valor de una variable que ha sido declarada pero no inicializada**.

También es el valor que **devuelven las funciones que no tienen return explícito**, o cuando accedes a una propiedad que no existe.

```
let x;
```

```
console.log(x); // undefined

function saludar() {
  console.log("Hola");
}

let resultado = saludar(); // "Hola"
console.log(resultado);    // undefined (no devuelve nada)

let persona = {};
console.log(persona.nombre); // undefined (no existe esa propiedad)
```

7.2. Null

null representa la ausencia intencional de valor. Es decir, tú lo asignas cuando quieres decir "esto está vacío" o "aún no hay valor aquí".

```
let usuario = null; // Todavía no hay usuario, pero esperamos uno más adelante

let datos = {
  nombre: "Ana",
  edad: null // intencionalmente sin edad
};

console.log(datos.edad); // null
typeof null // "object" ! (esto es un bug histórico del lenguaje)
```

7.3. Infinity

Infinity es un valor especial que representa el infinito matemático. Se produce, por ejemplo, cuando se divide entre cero.

```
console.log(1 / 0);    // Infinity
console.log(-1 / 0);   // -Infinity
console.log(Infinity + 1); // Infinity
```

7.4. Comparación entre null, undefined e Infinity

Valor	Tipo (typeof)	Significado común	Lo asigna...
undefined	"undefined"	Variable no inicializada	El lenguaje (JS)
null	"object" 🤪	Ausencia intencional de valor	Tú (como programador)
Infinity	"number"	Resultado matemático infinito	JavaScript (auto)

8. Conversión entre tipos de datos.

Aunque los tipos de datos en JavaScript son muy sencillos, a veces se podrá encontrar con casos en los que las operaciones no se realizan correctamente, y eso es debido a la conversión de tipos de datos. JavaScript intenta realizar la mejor conversión cuando realiza esas operaciones, pero a veces no es el tipo de conversión que a nosotros nos interesaría.

Por ejemplo, cuando intentamos sumar dos números:

```
4 + 5 // resultado = 9
```

Si uno de esos números está en formato de cadena de texto, JavaScript lo que hará es intentar convertir el otro número a una cadena y los concatenará, por ejemplo:

```
4 + "5" // resultado = "45"
```

Otro ejemplo podría ser:

```
4 + 5 + "6" // resultado = "96"
```

Esto puede resultar ilógico, pero sí que tiene su lógica. La expresión se evalúa de izquierda a derecha. La primera operación funciona correctamente devolviendo el valor de 9 pero al intentar sumarle una cadena de texto "6" JavaScript lo que hace es convertir ese número a una cadena de texto y se lo concatenará al comienzo del "6".

Para convertir cadenas a números dispones de las funciones: **parseInt()** y **parseFloat()**.

Por ejemplo:

```
parseInt("34") // resultado = 34  
parseInt("89.76") // resultado = 89
```

parseFloat devolverá un entero o un número real según el caso:

```
parseFloat("34") // resultado = 34  
parseFloat("89.76") // resultado = 89.76  
4 + 5 + parseInt("6") // resultado = 15
```

Si lo que se desea es realizar la conversión de números a cadenas, es mucho más sencillo, ya que simplemente tendrás que concatenar una cadena vacía al principio, y de esta forma el número será convertido a su cadena equivalente:

```
("" + 3400) // resultado = "3400"  
("" + 3400).length // resultado = 4
```

En el segundo ejemplo podemos ver la gran potencia de la evaluación de expresiones. Los paréntesis fuerzan la conversión del número a una cadena. Una cadena de texto en JavaScript tiene una propiedad asociada con ella que es la longitud (**length**), la cual te devolverá en este caso el número 4, indicando que hay 4 caracteres en esa cadena "3400". La longitud de una cadena es un número, no una cadena.

También podremos usar, como se verá más adelante, el método **toString(base)** que convierte un número a su representación en la base dada (valor numérico entre 2 y 36) como un string.

```
Var a = 5  
var b = a.toString() // b = "5"
```

9. Literales.

Los literales se utilizan para representar valores en JavaScript. Estos son valores constantes, no

variables. Nos podemos encontrar con los siguientes tipos de literales en JavaScript:

- Arrays de literales
- Literales lógicos [booleans]
- Literales de puntoflotante
- Literales enteros
- Objetos literales
- Cadenas literales (string)

9.1. Arrays de literales.

Un array literal es una lista de cero o más expresiones, cada una de las cuales representa un elemento del array, delimitada por corchetes ([]). Cuando se crea un array usando un array literal, aquel será inicializado con los valores especificados como sus elementos y su longitud será el número de argumentos especificados.

El siguiente ejemplo crea un array de cafés con tres elementos y una longitud de tres:

```
var cafés = ["Tueste francés", "Colombiano", "Kona"]; //Array de tamaño 3
var otroArray = ["Pepe", 40, true, 7.8, 8.9, 9.0, {curso:2DAW, nombre:"DAW"}];
```

9.2. Literales lógicos booleanos.

El tipo Boolean posee dos valores literales: true y false.

Tablas de verdad para los literales booleanos

X	Y	And
V	V	V
V	F	F
F	V	F
F	F	F

X	Y	OR
V	V	V
V	F	V
F	V	V
F	F	F

X	NOT (X)
V	F
F	V

Los operadores usados para el AND, OR y NOT en JavaScript son, respectivamente: "&&", "||" y "!".

9.3. Literales enteros

Los enteros pueden ser expresados en decimales (base 10), hexadecimales (base 16) y octales (base 8). Un literal entero decimal consiste en una secuencia de dígitos sin un 0 (cero) por delante. Un **0o** (cero+letra "o". La letra "o" puede estar también en mayúsculas) por delante en un literal entero indica que este es un octal; un **0x** (o **0X**) por delante indica que es un hexadecimal. Un entero hexadecimal puede incluir los dígitos (0-9) y las letras a-f y A-F. Los enteros octales pueden incluir solamente los dígitos 0-7. También podemos tener literales en binario, para ello basta con anteponer a la cifra en binario **0(cero)b** o **0(cero)B**.

Algunos ejemplos de literales enteros son:

```
0, 117 y -345 (decimales, base 10)
0o15, 000o1 y -0o77 (octales, base 8)
0x1123, 0x00111 y -0xF1A7 (hexadecimales, "hex" o base 16)
```

`0b10011`, `0B1010101` (binarios, base 2)

9.4. Literales de coma flotante.

Un literal de coma flotante puede contener las siguientes partes:

- Un entero decimal el cual puede tener signo (precedido por "+" o "-"),
- Un punto decimal ("."),
- Una fracción (otro número decimal),
- Un exponente.

La parte exponencial es una "e" o "E" seguida de un entero, el cual puede tener signo (precedido por "+" o "-"). Un literal de coma flotante debe tener al menos un dígito y, o un punto decimal o una "e" (o "E"). Algunos ejemplos de literales de coma flotante son:

`3.1415`

`-3.1E12`

`.1e12`

`2E-12`

De forma breve, la sintaxis es: [dígitos][.dígitos][(E|e)[(+|-)]dígitos].

9.5. Literales de objetos.

Un literal de objeto es una lista de cero o más pares de nombres de propiedades y valores asociados a dicho objeto, delimitada por llaves (`{}`). No se debe utilizar un literal de objeto al inicio de una sentencia. Esto provocaría un error o un comportamiento inesperado, debido a que una "`{`" será interpretada como el comienzo de un bloque.

El siguiente es un ejemplo de un literal de objeto. El primer elemento del objeto **automovil** define una propiedad, **miAuto**; el segundo elemento, la propiedad **obtenerAuto** invoca a la función (`TipoAuto("Honda")`); el tercer elemento, la propiedad especial usa una variable existente (`Ventas`).

```
var Ventas = "Toyota";

function TipoAuto(nombre)
{
    if(nombre == "Honda")
        return nombre;
    else
        return "Lo siento, no vendemos " + nombre + ".";
}
```

```
automovil = {miAuto: "Saturn", obtenerAuto: TipoAuto("Honda"), especial:
Ventas};
```

```
console.log (automovil.miAuto); // Saturn
console.log (automovil.obtenerAuto); // Honda
console.log (automovil.especial); // Toyota
```

Adicionalmente, se puede usar literales de cadena o numéricos para el nombre de una propiedad o para

anidar **un objeto dentro de otro**. El siguiente ejemplo utiliza estas opciones:

```
var car = {manyCars: {a: "Saab", b: "Jeep"}, 7: "Mazda"};

console.log(car.manyCars.b); // Jeep
console.log(car[7]); // Mazda
```

9.6. Literales de cadenas de caracteres (string).

Una cadena literal tiene cero o más caracteres encerrados entre comillas dobles (") o simples ('). Una cadena debe estar delimitada por comillas del mismo tipo, es decir, tanto las comillas simples como dobles al inicio y al final de la cadena. Los siguientes ejemplos son cadenas literales:

```
"blah"
'blah'
"1234"
"una línea \n otra línea"
"El gato de Juan"
```

Se puede invocar cualquiera de los métodos del objeto String (cadena) en un valor literal de cadena, JavaScript automáticamente la convierte de una cadena literal a un objeto String temporal, mediante la invocación del método, luego descarta el objeto String temporal. También se puede usar la propiedad **String.length** con una cadena literal:

```
"El gato de Juan".length;
```

9.7. Uso de caracteres especiales en cadenas de caracteres.

Además de los caracteres ordinarios, se puede también incluir caracteres especiales en las cadenas, tal como se muestra en el siguiente ejemplo.

```
"una línea \n otra línea"
```

Tabla. Caracteres especiales de JavaScript

Carácter	Significado
\b	Retroceso (Backspace)
\f	Avance de página
\n	Nueva línea
\r	Retorno de línea
\t	Tabulador [Tab]
\v	Tabulador vertical
\'	Apóstrofe o comilla simple

<code>\"</code>	Doble comilla
<code>\\</code>	Caracter Backslash (\).

9.8. Caracteres de escape.

Se puede insertar una comilla dentro de una cadena precediéndola por una contrabarra `"\"`. Esto se conoce como escapar la comilla. Por ejemplo:

```
var texto = "El lee \"La Incineración de Sam McGee\" de R.W. Service."  
console.log(texto)
```

El resultado de ello sería:

El lee "La Incineración de Sam McGee" de R.W. Service.

Otra opción es usar distintas comillas (dobles y simples) para abrir y cerrar el string y para que aparezcan dentro del string:

```
var texto = "El lee 'La Incineración de Sam McGee' de R.W. Service."  
console.log(texto)
```

El resultado de ello sería:

El lee 'La Incineración de Sam McGee' de R.W. Service.

Para incluir el literal correspondiente a la contrabarra [backslash] dentro de una cadena, deberá escapar el carácter de contrabarra. Por ejemplo, para asignar la ruta de un archivo `c:\temp` a una cadena, use lo siguiente:

```
var home = "c:\\temp"
```

10. Operadores.

JavaScript es un lenguaje rico en operadores: símbolos y palabras que realizan operaciones sobre uno o varios valores, para obtener un nuevo valor.

Cualquier valor sobre el cual se realiza una acción (indicada por el operador), se denomina un operando. Una expresión puede contener un operando y un operador (denominado operador unario), como por ejemplo en `b++`, o bien dos operandos, separados por un operador (denominado operador binario), como por ejemplo en `a + b`.

Tabla Categorías de los operadores en JavaScript

Tipo	Qué realizan
Comparación	Comparan valores de dos operandos, devolviendo el resultado de true o false. <code>==</code> , <code>!=</code> , <code>===</code> , <code>!==</code> , <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> .
Aritméticos	Unen dos operandos produciendo un único valor, resultado de una operación aritmética. <code>*</code> , <code>-</code> , <code>+</code> , <code>/</code> , <code>%</code> , <code>++</code> , <code>--</code> , <code>+valor</code> , <code>-valor</code> .

Asignación	Asigna el valor a la derecha de la expresión a la variable de la izquierda. =, +=, -=, *=, /=, %=, etc. Desde la versión JS 2021, tenemos estos otros operadores: • ??= → Asigna un valor a “nombre” solo si vale null/undefined. <pre>function saluda(nombre) { nombre ??= "Alumno"; // si no tiene valor (null o undefined), valdrá "Alumno" alert("Hola " + nombre); }</pre> <pre>saluda(); saluda("Pepe");</pre> • &&= → Asigna un valor a la variable si su valor se resuelve a true. <pre>var x = 1; var y = 0; x &&= "ok"; // x valdrá "ok", ya que su valor actual (1) resuelve (equivale) a true y &&= "not ok";</pre> • = → Asigna un valor a la variable si su valor se resuelve a false. <pre>var x = 1; var y = 0; x = "ok"; // x valdrá 1, ya que su valor actual (1) resuelve (equivale) a true y = "not ok";</pre>
Boolean	Realizan operaciones booleanas aritméticas sobre uno o dos operandos booleanos. && !
Bit a Bit	Operaciones aritméticas o de desplazamiento. &, , ^, ~<<, >>, >>>
Objeto	Evalúan la herencia y capacidades de un objeto. ., [], (), delete, in, instanceof, new, this
Miscelánea	Operadores con comportamiento especial. “,”, ?, :, typeof, void.

10.1. Operadores de comparación.

Tabla. Operadores de comparación

Sintaxis	Nombre	Tipos	Resultados
==	Igualdad	Todos	Boolean
!=	Distinto	Todos	Boolean
===	Igualdad estricta	Todos	Boolean
!==	Desigualdad estricta	Todos	Boolean
>, <	Mayor y menor que	Todos	Boolean

<code>>=</code>	Mayor o igual que	Todos	Boolean
<code><=</code>	Menor o igual que	Todos	Boolean

Por ejemplo:

```
30 == 30 // true
30 == 30.0 // true
5 != 8 // true
9 > 13 // false
7.29 <= 7.28 // false
```

También podríamos comparar cadenas a este nivel:

```
"Marta" == "Marta" // true
"Marta" == "marta" // false
"Marta" > "marta" // false
"Mark" < "Marta" // true
```

Para poder comparar cadenas, JavaScript convierte cada carácter de la cadena de un string, en su correspondiente valor ASCII. Cada letra, comenzando con la primera del operando de la izquierda, se compara con su correspondiente letra en el operando de la derecha. Los valores ASCII de las letras mayúsculas, son más pequeños que sus correspondientes en minúscula, por esa razón "Marta" no es mayor que "marta". En JavaScript hay que tener muy en cuenta la sensibilidad a mayúsculas y minúsculas.

Si por ejemplo comparamos un número con su cadena correspondiente:

```
"123" == 123 // true
true == 1 // true
```

JavaScript, cuando realiza esta comparación, convierte la cadena en su número correspondiente y luego realiza la comparación. También dispones de otra opción, que consiste en convertir mediante las funciones **parseInt()** o **parseFloat()** el operando correspondiente:

```
parseInt("123") == 123 // true
```

Los operadores `===` y `!==` comparan tanto el dato como el tipo de dato. El operador `===` sólo devolverá true, cuando los dos operandos son del mismo tipo de datos (por ejemplo, ambos son números) y tienen el mismo valor.

10.2. Operadores aritméticos.

Tabla. Operadores de aritméticos

	Nombre	Tipos de Resultados operandos	
+	Más	integer, float, string	integer, float, string

-	Menos	integer, float	integer, float
*	Multiplicación	integer, float	integer, float
/	División	integer, float	integer, float
%	Módulo	integer, float	integer, float
++	Incremento	integer, float	integer, float
--	Decremento	integer, float	integer, float
+valor	Positivo	integer, float, string	integer, float
-valor	Negativo	integer, float, string	integer, float

Algunos ejemplos:

```
var a = 10; // Inicializamos a al valor 10
var z = 0; // Inicializamos z al valor 0
z = a; // a es igual a 10, por lo tanto z es igual a 10.
z = ++a; // el valor de a se incrementa justo antes de ser asignado a z, por lo que a
es 11 y z //valdrá 11.
z = a++; // se asigna el valor de a (11) a z y luego se incrementa el valor de a (pasa
a ser 12).
z = a++; // a vale 12 antes de la asignación, por lo que z es igual a 12; una vez
hecha la //asignación a valdrá 13.

var x = 2;
var y = 8;
var z = -x; // z es igual a -2, pero x sigue siendo igual a 2.
z = - (x + y); // z es igual a -10, x es igual a 2 e y es igual a 8. z = -x + y;
// z igual a 6, pero x //es igual a 2 e y igual a 8.
```

10.3. Operadores de asignación.

Un operador de asignación asigna un valor a su operando izquierdo basándose en el valor de su operando derecho. El operador básico de asignación es el igual (=), el cual asigna el valor de su operador derecho a su operador izquierdo. Esto es, x=y asigna el valor de y a x.

Tabla. Operadores de asignación

Nombre		Ejemplo	Significado
=	Asignación	x=y	X=y
+=	Sumar un valor	x+=y	x=x+y

<code>-=</code>	Substraer un valor	<code>x-=y</code>	<code>x=x-y</code>
<code>*=</code>	Multiplicar un valor	<code>x*=y</code>	<code>x=x*y</code>
<code>%=</code>	Módulo de un valor	<code>x%=y</code>	<code>x=x%y</code>
<code><<=</code>	Desplazar bits a la izquierda	<code>x<<=y</code>	<code>x=x<<y</code>
<code>>>=</code>	Desplazar bits a la derecha	<code>x>>=y</code>	<code>x=x>>y</code>
<code>&=</code>	Operación AND bit a bit	<code>x&=y</code>	<code>x=x&y</code>
<code>! =</code>	Operación OR bit a bit	<code>x =y</code>	<code>x=x y</code>
<code>^=</code>	Operación XOR bit a bit	<code>x^=y</code>	<code>x=x^y</code>
<code>[]=</code>	Desestructuración de asignaciones	<code>[a,b]=[c,d]</code>	<code>a=c,b=d</code>

10.4. Operadores booleanos.

Debido a que parte de la programación tiene un gran componente de lógica, es por ello, que los operadores booleanos juegan un gran papel.

Los operadores booleanos te van a permitir evaluar expresiones, devolviendo como resultado `true` (verdadero) o `false` (falso).

Tabla. Operadores de asignación

Sintaxis	Nombre	Operandos	Resultados
<code>&&</code>	AND	Boolean	Boolean
<code> </code>	OR	Boolean	Boolean
<code>!</code>	Not	Boolean	Boolean

Ejemplos:

```

!true // resultado = false
!(10 > 5) // resultado = false
!(10 < 5) // resultado = true
!("gato" == "pato") // resultado = true
5 > 1 && 50 > 10 / resultado = true
5 > 1 && 50 < 10 / resultado = false
5 < 1 && 50 > 10 / resultado = false
5 < 1 && 50 < 10 / resultado = false

```

/

Tabla de verdad del operador AND

Operando	Operador AND	Operando	Resultado
True	&&	True	True
True	&&	False	False
False	&&	True	False
False	&&	False	False

Tabla de verdad del operador OR

Operando	Operador OR	Operando	Resultado
True		True	True
True		False	True
False		True	True
False		False	False

Ejemplos:

```

5 > 1 || 50 > 10 // resultado = true
5 > 1 || 50 < 10 // resultado = true
5 < 1 || 50 > 10 // resultado = true
5 < 1 || 50 < 10 // resultado = false

```

10.5. Operadores bit a bit.

Para los programadores de scripts, las operaciones bit a bit suelen ser un tema avanzado. Los operandos numéricos, pueden aparecer en JavaScript en cualquiera de los tres formatos posibles (decimal, octal o hexadecimal). Tan pronto como el operador tenga un operando, su valor se convertirá a representación binaria (32 bits de longitud). Las tres primeras operaciones binarias bit a bit que podemos realizar son AND, OR y XOR y los resultados de comparar bit a bit serán:

Bit a bit AND: 1 si ambos dígitos son 1.

Bit a bit OR: 1 si cualquiera de los dos dígitos es 1.

Bit a bit XOR: 1 si sólo un dígito es 1.

Tabla de operador bit a bit en JavaScript

Operador	Nombre	Operador derecho	Operador izquierdo
----------	--------	------------------	--------------------

&	Desplazamiento AND	Valor integer	Valor integer
	Desplazamiento OR	Valor integer	Valor integer
^	Desplazamiento XOR	Valor integer	Valor integer
~	Desplazamiento NOT	(ninguno)	Valor integer
<<	Desplazamiento a la izquierda	Valor integer	Cantidad a desplazar
>>	Desplazamiento a la derecha	Valor integer	Cantidad a desplazar
>>>	Desplazam. derecha rellanando con 0	Valor integer	Cantidad a desplazar

Por ejemplo:

```
4 << 2 // resultado = 16
```

10.6. Operador Pipe (|>)

Es una adición significativa que facilita la escritura de código funcional en JavaScript. Este operador permite encadenar funciones de una manera más limpia y legible. En lugar de anidar funciones dentro de otras, el Pipe Operator (|>) permite pasar el resultado de una función como entrada a la siguiente. Por ejemplo:

```
const text = "Hello World";
const result = text |> listen(%) |> exclaim(%) |> uppercase(%)
```

En este ejemplo, las funciones listen, exclaim y uppercase se aplican en orden secuencial, mejorando la legibilidad del código. Esta característica aún está en discusión, pero se espera que sea incluida en la especificación final.

Otro ejemplo:

```
const numbers = [ 10 , 20 , 30 , 40 , 50 ];

const processingNumbers = numbers
  |> ( _ => _. map ( n => n / 2 )) // Reducir a la mitad cada número
  |> ( _ => _. filter ( n => n > 10 )); // Filtrar números menores o iguales a 10

console . log (processedNumbers); // [15, 20, 25]
```

10.7. Operadores de objeto.

El siguiente grupo de operadores se relaciona directamente con objetos. La mayor parte de ellos fueron implementados a partir de las primeras versiones de JavaScript, por lo que puede haber algún tipo de

incompatibilidad con navegadores antiguos.

- **.** (punto). El operador punto, indica que el objeto a su izquierda tiene o contiene el recurso a su derecha, como por ejemplo: objeto.propiedad y objeto.método().

Ejemplo con un objeto nativo de JavaScript:

```
var s = new String('rafa');
var longitud = s.length;
var pos = s.indexOf("fa"); // resultado: pos = 2
```

- **[]** (corchetes para enumerar miembros de un objeto).

Por ejemplo, cuando creamos un array:

```
var a = ["Santiago", "Coruña", "Lugo"];
```

Enumerar un elemento de un array:

```
a[1] = "Coruña";
```

Enumerar una propiedad de un objeto:

```
a["color"] = "azul";
```

- **delete** (para eliminar un elemento de una colección).

Por ejemplo, si consideramos:

```
var oceanos = new Array("Atlántico", "Pacífico", "Indico", "Ártico");
```

Podríamos hacer:

```
delete oceanos[2];
```

Esto eliminaría el tercer elemento del array ("Indico"), pero la longitud del array no cambiaría. Si intentamos referenciar esa posición oceanos[2] obtendríamos **undefined**.

- **in** (para inspeccionar métodos o propiedades de un objeto).

El operando a la izquierda del operador, es una cadena referente a la propiedad o método (simplemente el nombre del método sin paréntesis); el operando a la derecha del operador, es el objeto que estamos inspeccionando. Si el objeto conoce la propiedad o método, la expresión devolverá true.

Ejemplo:

```
"write" in document
```

o también

```
"defaultView" in document
```

- **instanceof** (para comprobar si un objeto es una instancia de un objeto nativo de JavaScript).

Ejemplo:

```
a = new Array(1,2,3);
a instanceof Array; // devolverá true.
```

- **new** (para acceder a los constructores de objetos incorporados en el núcleo de JavaScript).

Ejemplo:

```
var hoy = new Date(); // creará el objeto hoy de tipo Date() empleando el constructor por defecto de dicho objeto.
```

- **this** (para hacer referencia al propio objeto en el que estamos localizados).

Ejemplo:

```
nombre.onChange = validateInput;
function validateInput(evt) {
var valorDeInput = this.value; // Este this hace referencia al objeto nombre que estamos validando.
```

- **globalThis**

JavaScript se utiliza cada vez más en una amplia variedad de entornos. Además del navegador web, que es el tipo más común de entorno para JavaScript, puede ejecutar programas JavaScript en servidores, smartphones e incluso hardware robótico.

Cada entorno tiene su propio modelo de objeto y proporciona una sintaxis diferente para acceder al objeto **Global**, es decir, al elemento sobre el cual se ejecuta nuestro código JavaScript. En el navegador web, por ejemplo, se puede acceder al objeto *global* a través del objeto **window**, **self** o **frames**. En Node.js, sin embargo, estas propiedades no existen y debe usar *global* en su lugar. En **Web Workers** **, solo está disponible **self**.

Para solucionar esto, podemos usar [globalThis](#) para hacer siempre referencia al objeto *global* sin importar en que entorno estemos.

**** Web workers es una tecnología usada en HTML5 con la cual podremos hacer multithreading en JavaScript cuando estemos realizando tareas pesadas. Permite ejecutar scripts en segundo plano sin bloquear nuestra aplicación web. Los workers podrán comunicarse con el script principal y viceversa.**

10.8. Otros operadores.

- **El operador coma ,**

Este operador, indica una serie de expresiones que van a ser evaluadas en secuencia, de izquierda a derecha. La mayor parte de las veces, este operador se usa para combinar múltiples declaraciones e inicializaciones de variables en una única línea.

Por ejemplo:

```
var nombre, direccion, apellidos, edad;
```

Otra situación en la que podemos usar este operador coma, es dentro de la expresión **for**. En el siguiente ejemplo inicializamos dos variables de tipo contador, y las incrementamos en diferentes porcentajes. Cuando comienza el bucle, ambas variables se inicializan a 0 y a cada paso del bucle una de ellas se incrementa en 1, mientras que la otra se incrementa en 10.

```
for (let i=0, j=0 ; i < 125; i++, j+10) {
    // más instrucciones aquí dentro
}
```

- **? : (operador condicional)**

Este operador condicional es la forma reducida de la expresión `if .... else`. La sintaxis formal para este operador condicional es:

`condicion ? expresión si se cumple la condición: expresión si no se cumple;`

Si usamos esta expresión con un operador de asignación:

`var <identificador> = condicion ? expresión si se cumple la condición: expresión si no se cumple;`

Ejemplo:

```
var a,b;
a = 3; b = 5;
var h = a > b ? a : b; // a h se le asignará el valor 5;
```

- **typeof (devuelve el tipo de valor de una variable o expresión).**

Este operador unario se usa para identificar cuando una variable o expresión es de alguno de los siguientes tipos: `number`, `string`, `boolean`, `object`, `function` o `undefined`.

Ejemplo:

```
if (typeof miVariable == "number") {
    miVariable = parseInt(miVariable);
}
```

11. Bloques de código.

Un bloque de código es utilizado para agrupar sentencias. El bloque está delimitado por un par de llaves:

```
{
    sentencia_1
    sentencia_2
    .
    sentencia_n
}
```

Los bloques de sentencias son comúnmente utilizados con sentencias de flujo de control (en general [por ejemplo] `if`, `for`, `while`).

Ejemplo:

```
while (x < 10) {
    x++;
}
```

aquí, `{ x++; }` es un bloque de sentencias.

12. Decisiones.

En los lenguajes de programación, las instrucciones que te permiten controlar las decisiones y bucles de ejecución, se denominan "**Estructuras de Control**". Una estructura de control, dirige el flujo de ejecución a través de una secuencia de instrucciones, basadas en decisiones simples y en otros factores.

Una parte muy importante de una estructura de control es la "**condición**". Cada condición es una expresión que se evalúa a true o false.

En JavaScript tenemos varias estructuras de control, para las diferentes situaciones que te puedas encontrar durante la programación. Tres de las estructuras de control más comunes son: construcciones if, construcciones if...else y los bucles.

12.1. Construcción if.

La decisión más simple que podemos tomar en un programa, es la de seguir una rama determinada si una determinada condición es true.

Sintaxis:

```
if (condición) // entre paréntesis irá la condición que se evaluará a true o false.
{
    // instrucciones a ejecutar si se cumple la condición
}
```

Ejemplo:

```
if (miEdad > 30)
{
    alert("Ya eres una persona adulta");
}
```

12.2. Construcción if... else.

En este tipo de construcción, podemos gestionar que haremos cuando se cumpla y cuando no se cumpla una determinada condición.

Sintaxis:

```
if (condición) {
    // entre paréntesis irá la condición que se evaluará a true o false.
    // instrucciones a ejecutar si se cumple la condición
}
else {
    // instrucciones a ejecutar si no se cumple la condición
}
```

Ejemplo:

```
if (miEdad >30) {
    alert("Ya eres una persona adulta.");
}
else {
    alert("Eres una persona joven.");
}
```

12.3. Construcción switch.

En este tipo de construcción, se evalúa la expresión, y en función del valor que adopte, se ejecuta algunas de las alternativas prevista. Si la expresión no equivale a los patrones establecidos en los **case**, se ejecuta las sentencias establecidas por defecto. El **break** se utiliza para que una vez que entra en una opción, no se intente entrar en otras alternativas.

Sintaxis:

```
switch(Expresión) {  
  case Constante1:  
    //Bloque de sentencias1; break;  
  case Constante2:  
    //Bloque de sentencias2; break;  
  ...  
  case ConstanteN:  
    //Bloque de sentenciasN; break;  
  default:  
    //Bloque de sentencias por defecto;  
}
```

No es necesario poner **break** en el último **case**.

Si varias opciones **case** ejecutan el mismo código podemos unirlos de la siguientes forma:

```
switch(Expresión) {  
  case Constante1:  
    //Bloque de sentencias1; break;  
  case Constante2:  
  case Constante3:  
    //Bloque de sentenciasN; break;  
  default:  
    //Bloque de sentencias por defecto;  
}
```

En el ejemplo anterior, el **case** 2 y 3 ejecutarán el mismo código.

13. Bucles.

Los bucles son estructuras repetitivas, que se ejecutarán un número de veces fijado expresamente, o que dependerá de si se cumple una determinada condición.

13.1. Bucle for.

Este tipo de bucle te deja repetir un bloque de instrucciones un número limitado de veces.

Sintaxis:

```
for (expresión inicial; condición; incremento) {  
  // Instrucciones a ejecutar dentro del bucle.  
}
```

```
}
```

Ejemplo:

```
for (let i=1; i<=20; i++) {  
    // Instrucciones que se ejecutarán 20 veces.  
}
```

13.2. Bucle for.. in

Una estructura de control derivada de for es la estructura **for...in**. Su definición exacta implica el uso de **objetos**, que se estudiarán con detalle más adelante. Por tanto, solamente se va a presentar la estructura for...in adaptada a su uso en **arrays**. Su definición formal adaptada a los arrays es:

```
for(indice in array) { ...  
}
```

Si se quieren recorrer todos los elementos que forman un array, la estructura **for...in** es la forma más eficiente de hacerlo. Ejemplo:

```
dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];  
for(i in dias) {  
    console.log(dias[i]);  
}
```

La instrucción for-in itera sobre todas las propiedades enumerables de un objeto:

```
const object = { a: 1, b: 2, c: 3 };  
  
for (const property in object) {  
    console.log(`${property}: ${object[property]}`);  
}  
  
// expected output:  
// "a: 1"  
// "b: 2"  
// "c: 3"
```

13.3. Bucle for.. of

La **sentencia for...of** crea un bucle que itera a través de los elementos de **objetos** iterables (incluyendo Array, String, Map, el objeto arguments, etc.), ejecutando las sentencias de cada iteración con el valor del elemento que corresponda.

Ejemplo:

```
let iterable = [10, 20, 30];  
  
for (let value of iterable) {  
    value += 1;  
    console.log(value); // 11, 21, 31  
}
```

En el ejemplo anterior **value** obtiene directamente el valor del elemento sobre el que se itera, a diferencia del bucle **for.. in**, el cual obtiene solamente el índice.

El bucle **for.. of** es muy usado para acceder directamente a las propiedades de los objetos, que se verán más adelante. Aun así, mostramos otro ejemplo para diferenciar **for.. in** y **for.. of** iterando sobre objetos:

```
var listaPersonas= [
  {"nombre":"pepe","apellidos":"perez"},
  {"nombre":"ana","apellidos":"gomez"},
  {"nombre":"almudena","apellidos":"blanco"}
];

for (let persona in listaPersonas) {
  console.log(listaPersonas[persona].nombre;
  console.log(listaPersonas[persona].apellidos;
}

for (let persona of listaPersonas) {
  console.log(persona.nombre);
  console.log(persona.apellidos);
}
```

13.4. Bucle while.

Este tipo de bucles se utilizan cuando queremos repetir la ejecución de unas sentencias un número indefinido de veces, siempre que se cumpla una condición. Es más sencillo de comprender que el bucle **for**, ya que no incorpora en la misma línea la inicialización de las variables, su condición para seguir ejecutándose y su actualización. Sólo se indica, como veremos a continuación, la condición que se tiene que cumplir para que se realice una iteración o repetición.

Sintaxis:

```
while (condición) {
  // Instrucciones a ejecutar dentro del bucle.
}
```

Ejemplo:

```
var i=0;
while (i <=10) {
  // Instrucciones a ejecutar dentro del bucle hasta que i sea mayor que 10 y no
  // se cumpla
  //la condición. i++;
}
```

También podemos usar la estructura **do {} while**;

Ejemplo:

```
do {
  // código
}
while (condición);
```

14. Sentencias break y continue

La estructura de control *for* es muy sencilla de utilizar, pero tiene el inconveniente de que el número de repeticiones que se realizan sólo se pueden controlar mediante las variables definidas en la zona de actualización del bucle.

Las sentencias *break* y *continue* permiten manipular el comportamiento normal de los bucles *for* para detener el bucle o para saltarse algunas repeticiones. Concretamente, la sentencia *break* permite terminar de forma abrupta un bucle y la sentencia *continue* permite saltarse algunas iteraciones del bucle.

Ejemplo:

```
var cadena = "En un lugar de la Mancha de cuyo nombre no quiero acordarme...";
var letras = cadena.split("");
var resultado = "";

for (i in letras) {
    if (letras[i] == "a")
        break; // de igual forma se puede usar continue.
    else
        resultado += letras[i];
}
```

15. Funciones

Cuando se desarrolla una aplicación compleja, es muy habitual utilizar una y otra vez las mismas instrucciones. Cuando una serie de instrucciones se repiten una y otra vez, se complica demasiado el código fuente de la aplicación, ya que:

- El código de la aplicación es mucho más largo porque muchas instrucciones están repetidas.
- Si se quiere modificar alguna de las instrucciones repetidas, se deben hacer tantas modificaciones como veces se haya escrito esa instrucción, lo que se convierte en un trabajo muy pesado y muy propenso a cometer errores.

Las funciones son la solución a estos problemas. Una función es un conjunto de instrucciones que se agrupan para realizar una tarea concreta y que se pueden reutilizar fácilmente. Para crear una función en JavaScript se usa la palabra reservada **function**.

Ejemplo:

```
function suma_y_muestra() {
    resultado = num1 + num2;
    alert ("El resultado es " + resultado);
}
...

var resultado;
var num1 = 5;
var num2 = 8;
```

```
suma_y_muestra();
```

Una vez creada la función, se puede llamar desde cualquier punto del código. La llamada a la función se realiza simplemente indicando su nombre, incluyendo los paréntesis y el carácter “;” es opcional.

15.1. Argumentos y valores de retorno

Las variables que necesitan las funciones se llaman **argumentos**. Antes de que pueda utilizarlos, la función debe indicar cuántos argumentos necesita y cuál es el nombre de cada argumento. Además, al invocar la función, se deben incluir los valores que se le van a pasar a la función. Los argumentos se indican dentro de los paréntesis que van detrás del nombre de la función y se separan con una coma (,).

```
function suma_y_muestra(primerNum, segundoNum) {  
    resultado = primerNum + segundoNum;  
    alert (“El resultado es “ + resultado);  
}  
  
var resultado;  
var num1 = 5;  
var num2 = 8;  
  
suma_y_muestra(num1, num2);
```

Hay que tener cuidado con el orden de los parámetros actuales.

Una función puede devolver también valores. Para ello usaremos **return**.

Ejemplo:

```
function suma_y_muestra(primerNum, segundoNum) {  
    resultado = primerNum + segundoNum;  
    return resultado;  
}  
  
var valor;  
var num1 = 5;  
var num2 = 8;  
  
valor = suma_y_muestra(num1, num2);  
alert (“El resultado es “ + valor);
```

Una cosa que siempre ha tenido JavaScript, y que ha sido fruto de controversia entre los programadores de otros lenguajes, es que se puede definir una función con más parámetros que con los que se la llama.

Ejemplo:

```
function ejemplo (nombre, apellidos) {  
    if (apellidos != undefined){  
        console.log (“Hola “ + nombre + “ ” + apellidos);  
    }  
}
```

```
    } else {  
        console.log("Hola " + nombre);  
    }  
}  
  
ejemplo("Manuel");  
ejemplo("Pepito", "Grillo");
```

Pero ahora se pueden declarar valores por defecto para esos parámetros.

Ejemplo:

```
function ejemplo (nombre, apellidos = "García") {  
    console.log("Hola " + nombre + " " + apellidos);  
}  
  
ejemplo("Manuel");           //Hola Manuel García  
ejemplo("Pepito", "Grillo"); //Hola Pepito Grillo
```

En este ejemplo, si se llama a la función con un solo argumento, se coge por defecto “García” como segundo argumento. Otra opción es usar el operador “|” como se muestra en el ejemplo siguiente:

```
function ejemplo (valor1, valor2) {  
    let v1 = valor1 | 1;  
    let v2 = valor2 | 1;  
    return v1*v2;  
}  
  
ejemplo();    // devolverá 1  
ejemplo(35);  // devolverá 35  
ejemplo(3,67); // devolverá 201
```

15.2. Argumentos de una función: Array arguments

Dentro de una función podemos acceder a los parámetros con los que la hemos llamado a través del Array **arguments[]**. Este Array lo crea JavaScript de forma automática y tendrá los mismos elementos y en el mismo orden en el que se ha llamado a la función.

```
function muestraValores() {  
    for (let i in arguments)  
        document.write(arguments[i]);  
}  
  
muestraValores(1,2,3,4,5);
```

Este método nos permite llamar a una misma función varias veces con distinto número de parámetros.

15.3. Parámetro REST y operador SPREAD

En una función se permite que el último argumento que se le pasa, sea especial y se denota con **tres puntos suspensivos**. Eso nos indica que este último argumento (llamado REST), recibirá todos los parámetros de más de la función en forma de array:

```
function ejemplo (param1, param2, ...restParams) {
  console.log(restParams);
}
ejemplo('a','b','c','d','e','f'); //el resultado es el array ["c", "d", "e", "f"]
```

¿Y la operación contraria? Imaginemos que lo que tenemos es una función variádica que espera un número variable de parámetros en lugar de un array. Podemos convertir fácilmente el array a una lista de parámetros con el operador **SPREAD**, que también se denota con tres puntos suspensivos.

Esto puede parecer que da lugar a equívocos, pero fijémonos que uno se utiliza para definir una función, y el otro para invocarla.

```
let aVocales = ["a", "e", "i", "o", "u"];

function bar(){
  let txt = '';
  for(let i in arguments){
    txt += arguments[i];
  }
  console.log(txt); // la salida por consola es aeiou
}

function foo(a){
  console.log(a.length); //a es un array
  bar(...a); //esto es como llamar a bar('a','e','i','o','u') (PARÁMETRO SPREAD)
}

foo(aVocales);
```

15.4. Generadores

Procesar cada uno de los elementos en una colección es una operación muy común. JavaScript proporciona diversas formas de iterar a través de los elementos de una colección, desde simples bucles **for** hasta métodos como **map()** y **filter()**. Los generadores llevan directamente el concepto de iteración al centro del lenguaje y proporcionan un mecanismo para personalizar el comportamiento de los bucles **for...of**.

Una función generadora (constructor *GeneratorFunction*) es un tipo especial de función que sirve como una fábrica de iteradores. Cuando se ejecuta, regresa un nuevo objeto *Generator*. Una función se convierte en una Función Generadora si utiliza la sintaxis **function***.

```
function* counterGenerator() {
  let i = 0;
  while (i < 100) {
```

```
    yield 2 * i + 1; // Números impares
    i++;
  }
}

var counter = counterGenerator();

do {

  console.log(counter.next()); // { value: 1..., done: false }

} while (!counter.next().done);
```

Este sencillo ejemplo muestra el funcionamiento de un generador. El uso más habitual de los generadores es crear *Iteradores*. Un *Iterador* es un objeto que devuelve un elemento de una colección cada vez que llamamos a su método `.next()`.

Los generadores siempre devuelven un iterador y en el momento en el que llamamos al método `.next()` del iterador, éste ejecuta la función del generador hasta llegar al primer **yield** que encuentra, que detiene la ejecución de la función y *produce* un elemento de la colección.

El resultado es siempre un objeto con dos propiedades, **value** y **done** (`counter.next().value` y `counter.next().done` respectivamente), en la primera está el valor producido por **yield** y la segunda es para indicar si el iterador ha terminado, es decir, si ese era el último elemento de la colección.

15.5. Funciones anónimas

La mayoría de los tipos básicos (números y cadenas de caracteres principalmente) pueden integrarse en la lógica del código sin necesidad de ser almacenados en variables. Es decir, para sumar 2 y 3 podemos escribir el código `2 + 3` sin necesidad de guardar estos datos en variables. Pues bien, como las funciones son variables, podemos crear una donde deseemos usarla sin necesidad de darle un nombre. Esto es lo que se conoce como funciones anónimas.

```
function () {
  console.log('Esta función no tiene un nombre')
}
```

Aunque en ese ejemplo el código escrito no tiene mucho sentido, ya que no podemos llamar a una función sin nombre, se puede añadir un par de caracteres más para que resulte efectivo:

```
(function () {
  console.log('Esta función no tiene nombre y se ejecutará inmediatamente');
})();
```

Con estas líneas hemos hecho tres cosas:

1. Agrupar el código escrito entre paréntesis para que no haya errores sintácticos con el resto del código.
2. Crear una función anónima (sin nombre) con un código a ejecutar.
3. Colocar `()` inmediatamente después de esta función para decir a JavaScript “¡oye! Ejecuta este código que estoy poniendo aquí como si se tratara de una función”.

Con esta forma de crear funciones anónimas no es de extrañar que además se puedan mandar parámetros a la función sin ningún problema:

```
(function (uno, dos, tres) {
```

```
console.log(unos);
console.log(dos);
console.log(tres);
}(1, 2, 3));
```

Y finalmente surge una pregunta: **¿de qué sirve crear una función anónima?** Es cierto que parece una de esas feas costumbres que no hacen más que ofuscar el código y hacerlo más difícil de leer, pero al menos se pueden mencionar dos casos en los que crear una función anónima resulta de lo más práctico y mantiene el código limpio:

1. Queremos **pasar como parámetro una función muy sencilla** y definirla a parte sería cuanto menos innecesario. Esto puede darse por ejemplo cuando pasamos una función de comparación a una de ordenación o cuando usamos librerías como jQuery (muchos de sus métodos aceptan paso de funciones).
2. Intentamos evitar a toda costa el uso de variables globales. Aunque este caso puede evitarse organizando el código de muchas formas, englobar el **“main”** de nuestro programa en una función anónima nos **evita dejar esas tres o cuatro variables** que necesitamos para que todo empiece a andar flotando en la memoria **encapsulándolas dentro de la función**.

Si no añadimos los paréntesis finales, es como si asignáramos la función anónima a una variable, que será de tipo **function** a partir de ese momento. Esta variable a la que asignamos la función anónima podremos usarla como una función más pasándole argumentos. Aquí un ejemplo:

```
var func = function (name, age) {    // Función anónima
    return "Nombre de usuario: " + name + " y edad: " + age
}

document.getElementById("h21").innerHTML += func("Martina", 39)
```

15.6. Funciones arrow

Las funciones arrow (flecha) nos sirven para crear una sintaxis de llamadas a funciones más simple. Se denotan por el símbolo `=>` y se utilizan de la siguiente manera. Las siguientes expresiones son equivalentes:

```
let multiplicar = function (a,b) {
    return a * b;
}

let multiplicar = (a,b) => a * b;

console.log(multiplicar(3,5)); //15
```

La sintaxis es la siguiente:

- (param1, param2, ..., paramN) => { sentencias }
- (param1, param2, ..., paramN) => expresion // Equivalente a: () => { return expresion; }

Los paréntesis son opcionales cuando sólo dispone de un argumento: `singleParam => { sentencias }`

- (singleParam) => { sentencias } `singleParam => { sentencias }`

Una función sin argumentos requiere paréntesis:

- `() => { sentencias }`

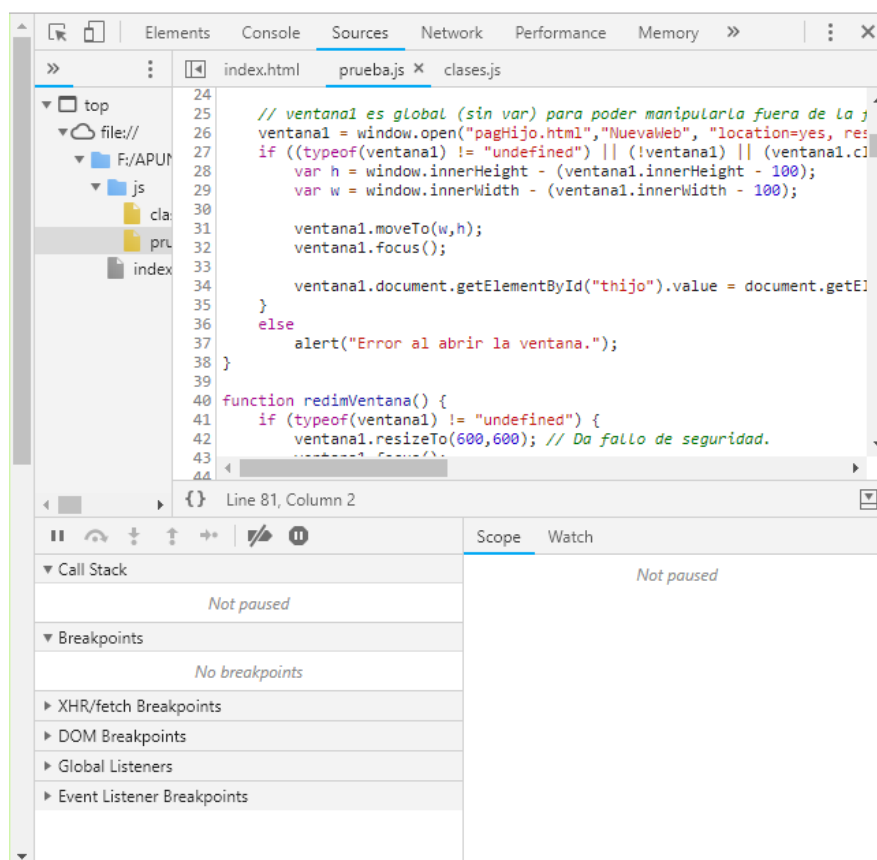
Estas funciones arrow no vinculan sus propios **this** o **arguments**. Se usan como una forma de llamar a una función más corta que usando la sintaxis convencional. **Las funciones arrow siempre son anónimas.**

16. Depuración de código JavaScript desde el navegador

Aprender a depurar nuestro código JavaScript desde los navegadores es una tarea fundamental para comprobar el correcto funcionamiento de nuestras aplicaciones web. En este documento se explican los pasos básicos para empezar a probar el código desde el navegador Google Chrome. Los depuradores de los demás navegadores son muy similares.

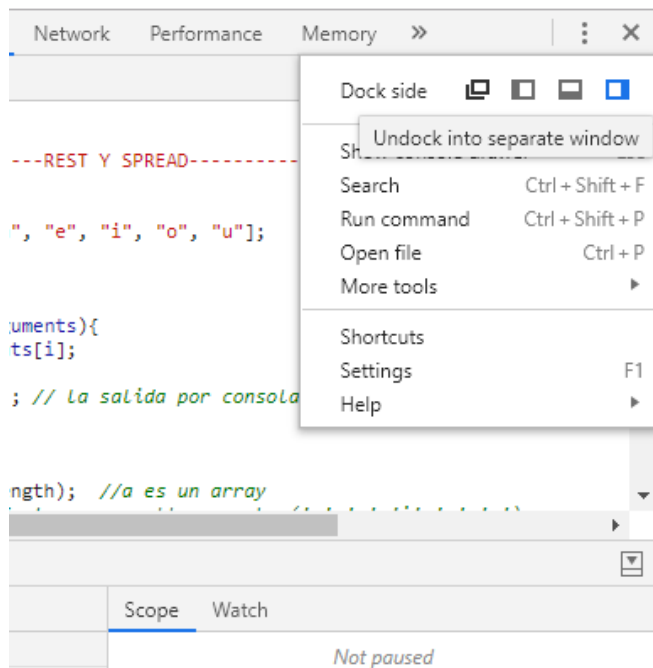
16.1. Abrir la consola de JavaScript

El navegador Google Chrome viene con herramientas de depuración de JavaScript integradas. Abrimos con él la página web que contiene el código *JavaScript* que nos interesa y pulsamos el botón de "personaliza y controla Google Chrome", unos puntitos que aparecen en la parte superior derecha del navegador. Se abre un menú y elegimos **"Herramientas"** -> **"Herramientas para desarrolladores"**. También se puede abrir rápidamente con la combinación de teclas **Ctrl + Mayus. + I**.



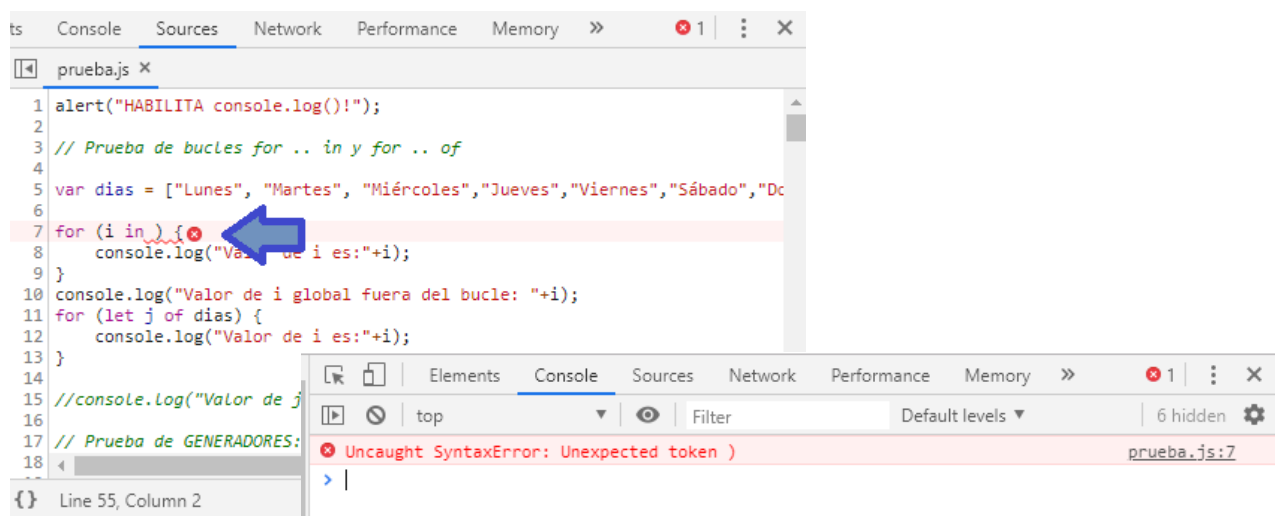
Según lo tengamos configurado puede partirnos el navegador en dos, dejando en la parte inferior

la *consola de javascript*, o bien puede abrirnos una ventana separada con la consola. Podemos separar o juntar dicha ventana pulsando el botón en la parte inferior izquierda cuyo tooltip pone "**undock into separate window**" o "**dock into main window**".



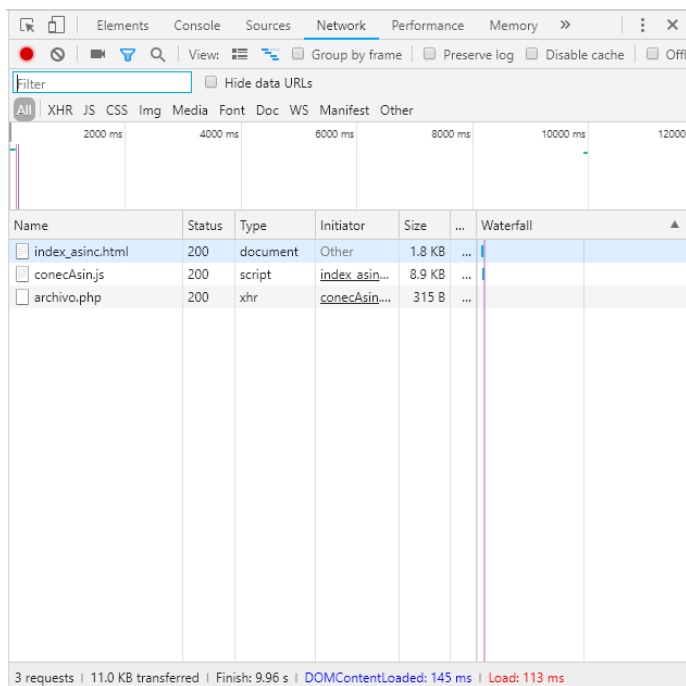
16.2. Visualizar los errores del código

Una vez abiertas las herramientas de desarrollo, podemos observar una serie de pestañas en la parte superior. En esta sección comentaremos la pestaña "**Console**". En ella podremos visualizar los distintos errores que nos van apareciendo en el código, así como el archivo y la línea que los ha producido. Solamente tenemos que recargar la página, o pulsar F5, para que el código se vuelva a ejecutar. Si aparecen errores se verán las líneas de código subrayadas de rojo y el error concreto junto con la línea donde se ha producido en **console**.



16.3. Comprobar el tráfico de red

La opción **"Network"** nos permitirá comprobar el tamaño de datos que se han transferido desde el servidor remoto hasta nuestro cliente y el tiempo que se ha tardado en transferir esos datos durante la ejecución de nuestra aplicación web.



Esta opción nos muestra el tamaño de cada uno de los recursos que se han transferido en una conexión desde el servidor. Estos recursos pueden ser scripts, funciones, ficheros, etc.

Además, en la parte inferior de la ventana nos muestra el tiempo que se ha tardado en realizar esta transferencia.

Esta opción es muy útil para comprobar la eficiencia de nuestra aplicación web a la hora de enviar y recuperar datos desde el servidor.

16.4. Escribir código JavaScript

En esta consola podemos escribir código JavaScript y ejecutarlo sin más que pulsar <intro>. Si nuestro código ocupa varias líneas, para empezar una nueva línea sin que se ejecute el código mantenemos pulsada la tecla <shift> al apretar <intro>. Para empezar una nueva línea al escribir pulsamos <shift><intro>, para ejecutar al final, simplemente <intro> obteniendo el resultado.

La consola tiene autocompletar, por lo que si vamos escribiendo alguna función de JavaScript o el nombre de alguna variable que existe, nos mostrará las posibles opciones según vamos escribiendo.

La consola, para mostrar resultados en pantalla, tiene el objeto **console** con el método log, de forma que podemos hacer

```
var a = ["a",2,false];  
console.log(a);
```

y nos mostrará el contenido de a. Esto es una alternativa más elegante a los típicos "alert" en nuestro código Javascript para depurar. Eso sí, console es algo que nos ofrece la consola de Google Chrome, por lo que no debemos eliminar esos log de nuestro código real cuando terminemos de depurar.

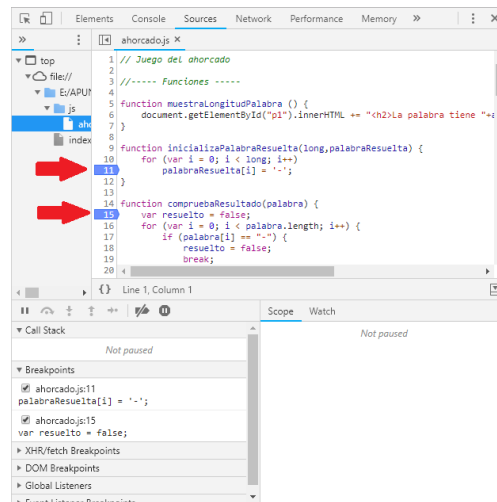
16.5. Depurar nuestro código javascript

Para depurar el código JavaScript de la página, pulsamos el botón **"Sources"** y en la parte izquierda, debajo del botón **"Elements"** aparece una pequeña flechita que debemos pulsar. se abrirá un listado con

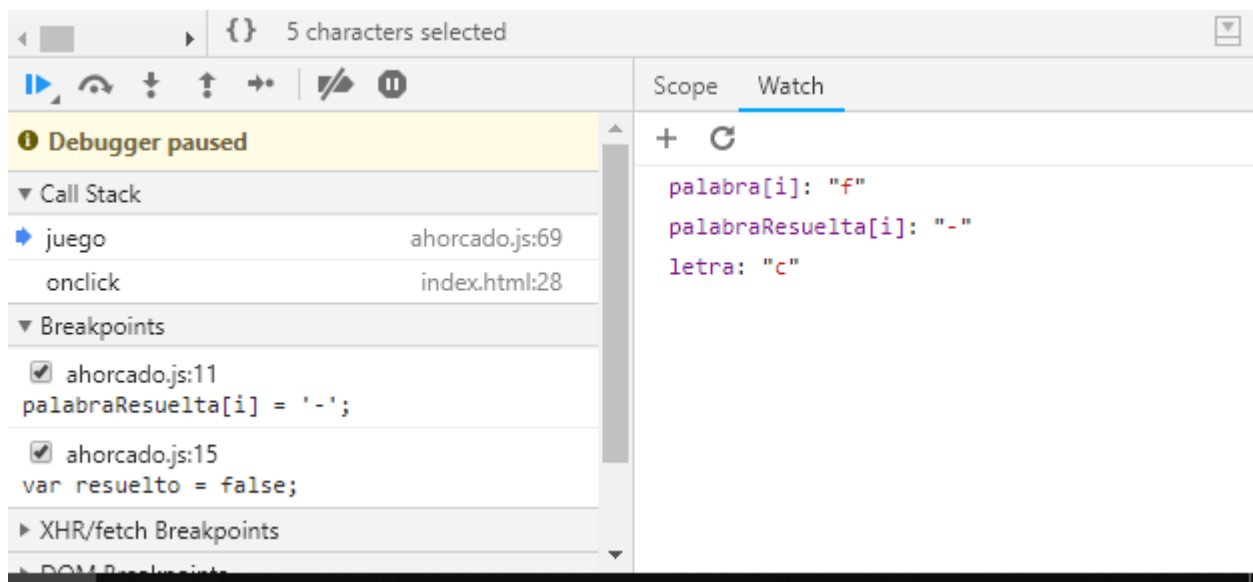
todos los ficheros Javascript que incluye la página. Seleccionando el de interés y veremos su código fuente.

16.6. Puntos de parada (breakpoints):

Podemos crear un “*breakpoint*” con solo pulsar en el número de línea donde queramos que se detenga la ejecución del código. Posteriormente volvemos a recargar el código para que se ejecuta desde el principio.



Una vez parado, en la parte derecha tenemos botones para avanzar paso a paso, entrando o saliendo de llamadas a funciones. Podemos añadir un “watch” para ver cualquier variable que nos interese y podemos ver cualquier variable definida. Si colocamos el ratón sobre una de las variables del código que esté ya definida, podemos ver su valor.



En la parte derecha tenemos varios botones que nos permiten ejecutar el código de distintas formas:



de izquierda a derecha son:

- **Ejecución hasta el siguiente punto de parada o hasta el final si no hay.**

- Ejecución paso a paso.
- Ejecución pasa a paso entrando en funciones.
- Ejecución paso a paso sin entrar en las funciones.
- Desactivar los puntos de parada.
- Pausar la ejecución del código.

En la parte derecha tenemos la opción “**Watch**” donde podremos visualizar el nombre de las variables del código y visualizar sus variables. La opción “**Scope**” nos muestra ya las variables que han ido apareciendo durante ejecuciones anteriores del código.

16.7. Alternativa a las Dev Tools

Si nuestro código no requiere por ahora de visualización en el navegador, por ejemplo, si estamos probando algunas funciones que hemos desarrollado que no devuelven contenido a la interfaz, podemos probar la salida de nuestro código .js en el terminal del sistema como si se tratase de un programa clásico en C, C++, C#, Java, ... Para ello tendremos que tener instalado **nodeJS** y **NPM**. Una vez instalado podemos ejecutar nuestro código usando:

```
node <fichero.js>
```

Con esto, la salida de mi código con **console.log(“cadena”)** o con **process.stdout.write(“cadena”)** saldrá por el terminal. Para leer datos del terminal introducidos por el usuario podemos usar **process.stdin.on(“data”, function(data) {<código de la función cuando se lea el dato>})**. Ejemplo:

```
process.stdout.write("Hola, ¿me dices tu nombre?\n");

process.stdin.on("data", function(data) {
  nombre = data.toString();
  process.stdout.write(`Hola ${nombre}`);

  process.exit(1); // Terminamos la ejecución.
});
```

Acabamos la ejecución con **process.exit**. Más info [aquí](#).

Hay una pequeña pega, y es que, si modificamos nuestro código, tendremos que volver a ejecutar node <fichero.js>. Para que node vuelva a ejecutar nuestro código automáticamente después de cada guardado podemos instalar la **nodemon**.

Nodemon es una utilidad que monitorea los cambios en el código fuente que se está desarrollando y automáticamente reinicia el servidor. Es una herramienta muy útil para desarrollo de aplicaciones en Node.js. La instalaremos de la siguiente forma:

```
npm install nodemon -g
```

Ahora solo debemos ejecutar nuestro código con **nodemon <fichero.js>**.

17. Ejecución de código JavaScript que accede a HTML

En muchas ocasiones ejecutaremos nuestros scripts JavaScript accediendo al contenido o estilo de los distintos elementos de HTML, por ejemplo, para escribir contenido en un párrafo (<p></p>). Para acceder a estos valores accederemos a las distintas etiquetas HTML por su nombre, tipo, clase o id.

Uno de los errores más comunes es que cuando nuestro script intenta acceder a una etiqueta HTML nos puede devolver el siguiente error:

Uncaught TypeError: Cannot read property 'innerHTML' of null

Se nos está informando que estamos intentando acceder a la propiedad `innerHTML` de un elemento que no existe. Esto sucede porque el código JavaScript se está ejecutando antes de que se carguen todas las etiquetas del HTML.

El código de JavaScript se suele poner (y es lo más recomendable) en la cabecera (`<head></head>`) del HTML. Por este motivo se ejecuta antes que la parte del `<body>`, donde se encuentran las etiquetas HTML.

Este problema podríamos solucionarlo si usáramos **jQuery**. Con jQuery se resuelve este problema introduciendo todo el código en su función `$(function() {})`;

Posibles soluciones:

Para empezar podemos filtrar si el valor que vamos a modificar de una etiqueta existe:

```
if (document.getElementById("p1").innerHTML != null)
    var aux = document.getElementById("p1").innerHTML;
```

Esto nos podrá ahorrar algunos posibles fallos pero no soluciona el problema. De forma similar, se puede preguntar por el elemento HTML en cuestión dentro de un bloque `try {} catch {}`.

Solución 1:

Podríamos enlazar el código JavaScript justo antes del `</body>`. No es muy eficiente ni profesional.

Solución 2:

```
document.addEventListener("DOMContentLoaded", function(event) {
    //Escribir código
});
```

En esta solución se crea una función anónima (sin nombre) que se ejecuta cuando se ha cargado todo el contenido HTML (contenido del árbol DOM). Todo lo que programemos dentro se ejecutará con el código HTML ya existente. Sustituiría a una función principal o main de programación clásica orientada a objetos.

Solución 3:

Otra posible solución para por ejecutar una función cuando se haya cargado todo el contenido de la ventana. Es algo parecido a la opción anterior, pero usando otro método:

```
window.onload = function miFuncion() {
    // Código...
};

window.onload = function() {
    // Código ...
};
```

Esta solución es similar a provocar el evento `onload` en el `<body>` del HTML:

```
<body onload="miFunción()">
```

Solución 4:

Dentro de la definición del código JavaScript y, dentro de la etiqueta `<script>`, se pueden referenciar los siguientes atributos:

- **src="..."**, indica la URL de un archivo externo que contiene el código del script.
- **type="text/javascript"**, especifica el tipo MIME (tipo de archivo/contenido). Ya no es necesario.
- **defer = "defer"**, indica al navegador que no pare la carga de la página por la ejecución del script. Ya no es necesario asignarle el valor **"defer"**, simplemente se especifica la etiqueta **defer**:

```
<script src="js/script.js" defer>
```

18. Try, catch(), finally, throw

Será muy conveniente crear código donde se capten los errores y nuestra aplicación pueda seguir funcionando e informando de los errores que se van obteniendo. Para ello disponemos en JavaScript de la estructura **try {} catch {} finally {}**. Podemos personalizar los errores que nos van apareciendo usando la estructura **throw**. Veamos un ejemplo:

```
try {
    if(x == "") throw "empty";
    if(isNaN(x)) throw "not a number";

    x = Number(x);

    if(x < 5) throw "too low";
    if(x > 10) throw "too high";
}

catch(err) {
    message.innerHTML = "Entrada es " + err;
}

finally {
    console.log("Seguimos...");
}
```

Cuando ejecutamos código Javascript, pueden ocurrir diferentes errores. Los errores pueden ser errores de codificación cometidos por el programador, errores debido a una entrada incorrecta y otras cosas imprevisibles.

Cuando ocurre un error, JavaScript normalmente se detendrá y generará un mensaje de error. En realidad, JavaScript lo que hace es crear un **objeto Error** con las propiedades **name** y **message**.

Con la declaración **throw** podremos crear errores personalizados y lanzar una excepción de error. La excepción puede ser de tipo **String**, **Number**, **Boolean** o un **Object**.

```
const message = document.getElementById("p1");
message.innerHTML = "";
let x = document.getElementById("demo").value;

var x = 0;

try {
    if(x === "") throw "empty";
```

```
if(isNaN(x)) throw "not a number";

if(x < 5) throw new Error("Numeric Error ", {cause: "too low" });
if(x > 10) throw new Error("Numeric Error ", {cause: "too low" });
}
catch(err) {
  message.innerHTML = (`${err}: ${err.cause}`);
}
```

En el ejemplo anterior puedes ver que, para los casos en que obtengamos un valor numérico menor que 5 o mayor que 10, se genera un objeto error añadiendo un campo “cause”. **Error.cause** permite especificar que causó un error.

ANEXO I: Funciones y propiedades básicas de JavaScript

JavaScript incorpora una serie de herramientas y utilidades (llamadas funciones y propiedades, como se verá más adelante) para el manejo de las variables. De esta forma, muchas de las operaciones básicas con las variables, se pueden realizar directamente con las utilidades que ofrece JavaScript.

Funciones útiles para cadenas de texto

A continuación se muestran algunas de las funciones más útiles para el manejo de cadenas de texto:

- **length**, calcula la longitud de una cadena de texto (el número de caracteres que la forman)

```
var mensaje = "Hola Mundo";
var numeroLetras = mensaje.length; // numeroLetras = 10
```

- **+**, se emplea para concatenar varias cadenas de texto

```
var mensaje1 = "Hola";
var mensaje2 = " Mundo";
var mensaje = mensaje1 + mensaje2; // mensaje = "Hola Mundo"
```

Además del operador **+**, también se puede utilizar la función **concat()**

```
var mensaje1 = "Hola";
var mensaje2 = mensaje1.concat(" Mundo"); // mensaje2 = "Hola Mundo"
```

Las cadenas de texto también se pueden unir con variables numéricas:

```
var variable1 = "Hola ";
var variable2 = 3;
var mensaje = variable1 + variable2; // mensaje = "Hola 3"
```

- **toUpperCase()**, transforma todos los caracteres de la cadena a sus correspondientes caracteres

en mayúsculas:

```
var mensaje1 = "Hola";  
var mensaje2 = mensaje1.toUpperCase(); // mensaje2 = "HOLA"
```

- **toLowerCase()**, transforma todos los caracteres de la cadena a sus correspondientes caracteres en minúsculas:

```
var mensaje1 = "HoIA";  
var mensaje2 = mensaje1.toLowerCase(); // mensaje2 = "hola"
```

- **charAt(posicion)**, obtiene el carácter que se encuentra en la posición indicada:

```
var mensaje = "Hola";  
var letra = mensaje.charAt(0); // letra = H  
letra = mensaje.charAt(2);    // letra = l
```

- **indexOf(caracter)**, calcula la posición en la que se encuentra el carácter indicado dentro de la cadena de texto. Si el carácter se incluye varias veces dentro de la cadena de texto, se devuelve su primera posición empezando a buscar desde la izquierda. Si la cadena no contiene el carácter, la función devuelve el valor -1:

```
var mensaje = "Hola";  
var posicion = mensaje.indexOf('a'); // posicion = 3  
posicion = mensaje.indexOf('b'); // posicion = -1 Su función análoga es lastIndexOf():
```

- **lastIndexOf(caracter)**, calcula la última posición en la que se encuentra el carácter indicado dentro de la cadena de texto. Si la cadena no contiene el carácter, la función devuelve el valor -1:

```
var mensaje = "Hola";  
var posicion = mensaje.lastIndexOf('a'); // posicion = 3  
posicion = mensaje.lastIndexOf('b'); // posicion = -1
```

La función lastIndexOf() comienza su búsqueda desde el final de la cadena hacia el principio, aunque la posición devuelta es la correcta empezando a contar desde el principio de la palabra.

- **substring(inicio, final)**, extrae una porción de una cadena de texto. El segundo parámetro es opcional. Si sólo se indica el parámetro inicio, la función devuelve la parte de la cadena original correspondiente desde esa posición hasta el final:

```
mensaje = "Hola Mundo";  
var porcion = mensaje.substring(2); // porcion = "la Mundo"  
porcion = mensaje.substring(5);    // porcion = "Mundo"  
porcion = mensaje.substring(7);    // porcion = "ndo"
```

Si se indica un inicio negativo, se devuelve la misma cadena original:

```
var mensaje = "Hola Mundo";  
var porcion = mensaje.substring(-2); // porcion = "Hola Mundo"
```

Cuando se indica el inicio y el final, se devuelve la parte de la cadena original comprendida entre

la posición inicial y la inmediatamente anterior a la posición final (es decir, la posición inicio está incluida y la posición final no):

```
var mensaje = "Hola Mundo";  
var porcion = mensaje.substring(1, 8); // porcion = "ola Mun"  
porcion = mensaje.substring(3, 4);    // porcion = "a"
```

Si se indica un final más pequeño que el inicio, JavaScript los considera de forma inversa, ya que automáticamente asigna el valor más pequeño al inicio y el más grande al final:

```
var mensaje = "Hola Mundo";  
var porcion = mensaje.substring(5, 0); // porcion = "Hola "  
porcion = mensaje.substring(0, 5);    // porcion = "Hola "
```

- **split(separador)**, convierte una cadena de texto en un array de cadenas de texto. La función parte la cadena de texto determinando sus trozos a partir del carácter separador indicado:

```
var mensaje = "Hola Mundo, soy una cadena de texto!";  
var palabras = mensaje.split(" ");  
// palabras = ["Hola", "Mundo,", "soy", "una", "cadena", "de", "texto!"];
```

Con esta función se pueden extraer fácilmente las letras que forman una palabra:

```
var palabra = "Hola";  
var letras = palabra.split(""); // letras = ["H", "o", "l", "a"]
```

Funciones útiles para arrays

A continuación, se muestran algunas de las funciones más útiles para el manejo de arrays:

- **length**, calcula el número de elementos de un array

```
var vocales = ["a", "e", "i", "o", "u"];  
var numeroVocales = vocales.length; // numeroVocales = 5
```

- **concat()**, se emplea para concatenar los elementos de varios arrays

```
var array1 = [1, 2, 3];  
array2 = array1.concat(4, 5, 6);    // array2 = [1, 2, 3, 4, 5, 6]  
array3 = array1.concat([4, 5, 6]);  // array3 = [1, 2, 3, 4, 5, 6]
```

- **join(separador)**, es la función contraria a split(). Une todos los elementos de un array para formar una cadena de texto. Para unir los elementos se utiliza el carácter separador indicado.

```
var array = ["hola", "mundo"];  
var mensaje = array.join(""); // mensaje = "holamundo"  
mensaje = array.join(" ");   // mensaje = "hola mundo"
```

- **pop()**, elimina el último elemento del array y lo devuelve. El array original se modifica y su longitud disminuye en 1 elemento.

```
var array = [1, 2, 3];
var ultimo = array.pop();
// ahora array = [1, 2], ultimo = 3
```

- **push()**, añade un elemento al final del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
var array = [1, 2, 3];
array.push(4);           // ahora array = [1, 2, 3, 4]
```

- **shift()**, elimina el primer elemento del array y lo devuelve. El array original se ve modificado y su longitud disminuida en 1 elemento.

```
var array = [1, 2, 3];
var primero = array.shift(); // ahora array = [2, 3], primero = 1
```

- **unshift()**, añade un elemento al principio del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
var array = [1, 2, 3];
array.unshift(0);           // ahora array = [0, 1, 2, 3]
```

- **reverse()**, modifica un array colocando sus elementos en el orden inverso a su posición original:

```
var array = [1, 2, 3];
array.reverse();           // ahora array = [3, 2, 1]
```

- **map()**, crea un nuevo array con los resultados de la llamada a la función indicada aplicados a cada uno de sus elementos.

```
var numbers = [1, 5, 10, 15];

var doubles = numbers.map(function(num) {
  return num * 2;
});

// doubles es ahora [2, 10, 20, 30]
// numbers sigue siendo [1, 5, 10, 15]
```

- **filter()**, se usa para filtrar los elementos de un array que cumplen una determinada condición.

```
let numbers = [1,5,23,4,12,45,78,8,8.9,10,11,3.4,10.1,84,6];
let numbersResult = [];

for (let i in numbers) {
  var currentNumber = numbers[i];
  if (currentNumber > 10) {
```

```
        numbersResult.push(currentNumber)
    }
}

console.log(numbersResult); // [23, 12, 45, 78, 11, 10.1, 84]
```

El anterior código se puede sustituir por el siguiente:

```
let numbers = [1,5,23,4,12,45,78,8,8.9,10,11,3.4,10.1,84,6]
let numbersResult = numbers.filter(num => {
    if (num > 10) return num;
});
```

Funciones útiles para números

A continuación, se muestran algunas de las funciones y propiedades más útiles para el manejo de números.

- **NaN**, (del inglés, "Not a Number"). JavaScript emplea el valor **NaN** para indicar un valor numérico no definido (por ejemplo, la división 0/0).

```
var numero1 = 0;
var numero2 = 0;
alert(numero1/numero2); // se muestra el valor NaN
```

- **isNaN()**, permite proteger a la aplicación de posibles valores numéricos no definidos.

```
var numero1 = 0;
var numero2 = 0;

if(isNaN(numero1/numero2)) {
    alert("La división no está definida para los números indicados");
}
else {
    alert("La división es igual a => " + numero1/numero2);
}
```

- **Infinity**, hace referencia a un valor numérico infinito y positivo (también existe el valor **-Infinity** para los infinitos negativos)

```
var numero1 = 10;
var numero2 = 0;
alert(numero1/numero2); // se muestra el valor Infinity
```

- **toFixed(digitos)**, devuelve el número original con tantos decimales como los indicados por el parámetro **digitos** y realiza los redondeos necesarios. Se trata de una función muy útil por ejemplo para mostrar precios.

```
var numero1 = 4564.34567;
numero1.toFixed(2); // 4564.35
```

```
numero1.toFixed(6); // 4564.345670  
numero1.toFixed(); // 4564
```

Funciones útiles para mostrar valores en un documento HTML

- **alert (“mensaje”)**

Este método tiene la desventaja de que habrá que pulsar el botón aceptar para pasar a mostrar el siguiente mensaje o dejar que se siga interpretando el código.

- **prompt (“mensaje”)**

Este método nos permite mostrar un mensaje y que el usuario meta un valor que podremos recoger en una variable en JavaScript.

- **console.log (“mensaje”)**

Este tipo de texto aparece solamente en la consola de los depuradores de los navegadores web. Muy útil para mostrar mensajes de trazas para probar el código que se está desarrollando.

- **document.write(“mensaje”) y document.writeln(“mensaje”)**

Muestra el mensaje directamente en la web. Está pensado para que el mensaje contenga una cadena de texto que incluya etiquetas HTML que se interpretarán con el formato que se ha definido. Para que funcione correctamente, deberemos ejecutarlo entre `document.open()` y `document.close()`. **Los navegadores impiden que se ejecute si el fichero .js no proviene del mismo dominio que index.html.**

```
document.open();  
document.write("<h1>Holaa!!</h1>")  
document.close();
```

- **document.confirm(“mensaje”)**

Permite mostrar un mensaje de forma similar al alert y además permite que el usuario elija si aceptar o no el mensaje. Se usa para realizar preguntas al usuario y seguir la ejecución según su decisión. Este método devuelve true o false.

- **innerHTML**

Esta propiedad permite insertar valores entre la etiqueta de cierre y de apertura de una etiqueta HTML.

ANEXO II: Conjuntos en JavaScript

Un conjunto es similar a un Array , excepto que cada valor solo se puede almacenar una vez. Por ejemplo, podemos tomar una lista de elementos, agregarlos todos a un conjunto y luego inspeccionar los resultados del conjunto. La lista de la derecha es el contenido de la lista de la izquierda, pero convertida en un conjunto. Hemos eliminado todos los duplicados de la lista porque tenemos la garantía de que un conjunto es único:

My list

```
1. apple
2. pear
3. carrot
4. pear
5. apple
6. yoshi
```

My set

```
1. apple
2. pear
3. carrot
4. yoshi
```

Puede que esto no parezca gran cosa en un ejemplo pequeño como este, pero es muy conveniente tener una forma integrada de crear colecciones únicas de elementos, especialmente con datos más grandes y tipos de datos y objetos más complejos. Por ejemplo, considere que puede agregar elementos a un conjunto de la siguiente manera:

```
const dogs = new Set();
const yoshi = { name: "Yoshi", personality: "Friendly" };
dogs.add(yoshi);
```

También **suele ser más rápido comprobar si un elemento está en un conjunto en lugar de en una matriz**, por lo que se adapta a los casos de uso en los que es necesario controlar el rendimiento en conjuntos de datos más grandes. También puede crear nuevos conjuntos con propiedades lógicas específicas basadas en conjuntos existentes. A continuación, veremos algunos ejemplos de ello.

Unión de dos conjuntos

Este es un buen ejemplo para empezar si no estás familiarizado con los conjuntos. Con la unión, podemos comprobar qué elementos están en cualquiera de los conjuntos o en ambos. En el siguiente ejemplo, tenemos dos listas y queremos crear una tercera que contenga todos los elementos de ambas listas, pero sin duplicados:

```
// Create a union of both sets
const unionSet = set1.union(set2);

// List the items in the union
unionSet.forEach((item) => {
  const li = document.createElement("li");
  li.textContent = item;
  unionList.appendChild(li);
});
```

Establecer intersección

Con la intersección de conjuntos, podemos comprobar qué elementos aparecen en ambos conjuntos para ver cuál es la superposición. En este ejemplo, en lugar de mostrar el conjunto de intersección como una tercera lista como hicimos anteriormente union, podemos usarlo para resaltar elementos que son "solo de ambos conjuntos":

```
// Make the intersection set
const intersectionSet = set1.intersection(set2);

// Loop through lists and highlight if they're in the intersection
allListItems.forEach((item) => {
  if (intersectionSet.has(item.textContent)) {
    item.className = "match";
  }
});
```

```
}  
});
```

Establecer diferencia simétrica

Antes de analizar la diferencia, veamos `symmetricDifference` que puede parecer complejo, pero esperamos que el siguiente ejemplo lo desmitifique. Lo que hace la diferencia simétrica es permitirnos verificar qué elementos están en cada conjunto, pero **no en ambos**.

```
const setNotBoth = set1.symmetricDifference(set2);  
  
allListItems.forEach((item) => {  
  if (setNotBoth.has(item.textContent)) {  
    item.className = "match";  
  }  
});
```

Diferencia de conjuntos

Con la diferencia de conjuntos, podemos comprobar qué elementos están en un conjunto pero no en otro. Para este ejemplo, estamos creando dos nuevos conjuntos en lugar de uno. El primero (`set1only`) se crea con `set1.difference(set2)` y lo que obtenemos es un nuevo conjunto de elementos que son "solo el conjunto uno". Hacemos lo mismo con el nuevo conjunto `set2only` para encontrar elementos que están en el conjunto dos pero no en el primero. Para cada lista, podemos usar los conjuntos creados con `difference` para resaltar los elementos de la lista que no aparecen en la otra lista.

```
const set1only = set1.difference(set2);  
const set2only = set2.difference(set1);  
  
allListItems.forEach((item) => {  
  if (set1only.has(item.textContent)) {  
    item.className = "setOneMatch";  
  } else if (set2only.has(item.textContent)) {  
    item.className = "setTwoMatch";  
  }  
});
```

ANEXO II: tipo Symbol.

Un Symbol es un **tipo de dato primitivo** (como string, number, etc.) que representa un **identificador único**. Aunque dos Symbol tengan la misma descripción, **nunca serán iguales**. Sirven para crear claves únicas para propiedades de objetos, evitando colisiones con otras claves.

¿Para qué sirven en la práctica?

Se usan cuando necesitas:

- Claves únicas que **no choquen** con otras propiedades.
- Añadir propiedades "privadas" u ocultas a objetos (que no se vean fácilmente al hacer `for...in` o `Object.keys()`).
- Usar ciertas funcionalidades avanzadas como `Symbol.iterator`, `Symbol.toPrimitive`, etc.



Ejemplo práctico: Evitar colisiones en un objeto

```
// Clave pública común
const persona = {
  nombre: "Ana",
  edad: 30
};

// Queremos añadir una propiedad especial sin riesgo de sobrescribir algo
const idUnico = Symbol("id");
persona[idUnico] = 12345;

// Veamos el resultado
console.log(persona); // { nombre: 'Ana', edad: 30, [Symbol(id)]: 12345 }

// Intentamos recorrer las claves
for (let clave in persona) {
  console.log(clave); // Solo muestra "nombre" y "edad"
}

console.log(Object.keys(persona)); // [ 'nombre', 'edad' ]
console.log(Object.getOwnPropertySymbols(persona)); // [ Symbol(id) ]
```

¿Qué pasó aquí?

- Creamos una **propiedad oculta** con un Symbol.
- Al hacer un for...in o Object.keys(), esa propiedad **no aparece**.
- Solo puedes acceder a ella si conoces el Symbol o usas Object.getOwnPropertySymbols().

Casos de uso avanzados con símbolos bien conocidos

JavaScript define **símbolos globales especiales** para personalizar el comportamiento de objetos:

Símbolo	Función
Symbol.iterator	Permite que un objeto sea iterable (por for...of)
Symbol.toPrimitive	Define cómo convertir el objeto a string, número, etc.
Symbol.toStringTag	Define el resultado de Object.prototype.toString.call(obj)

En resumen:

- Symbol crea identificadores **únicos e inmutables**.
- Son ideales para **propiedades privadas** o protegidas en objetos.
- Tienen aplicaciones muy útiles en el diseño de APIs, librerías y programación avanzada.